QUALCOMM AI RESEARCH (QARE) PREP • EYE-CARE WARM PARCHMENT EDITION

Pillar 5: Transformers, LLMs & Efficient Edge AI — Intuitive Study Textbook

Self-Attention, GQA, RoPE, SwiGLU, MoE • Roofline, KV-Cache, 16Ψ VRAM, FlashAttention • LoRA/QLoRA, RLHF/DPO/GRPO, Quantization & Speculative Decoding (LLM01–LLM20)

INTERACTIVE DIRECTORY • CLICK ANY TOPIC TO JUMP

Table of Contents (Mục Lục)

26 Sections Indexed

1. The Transformer Architecture from First Principles ..... p. 2

1.1 Why Self-Attention Replaced Recurrence ..... p. 2

1.2 Scaled Dot-Product Attention & The $\sqrt{d_k}$ Variance Proof (LLM01) ..... p. 3

1.3 Multi-Head Attention (MHA): Shapes, Parameters & The $N = 2D$ Crossover (LLM02 , LLM03) ..... p. 5

1.4 Causal Masking, Pre-LN vs Post-LN & Encoder vs Decoder Paradigms (LLM04) ..... p. 6

2. Modern LLM Architectural Upgrades (LLaMA / Qwen / Mistral DNA) ..... p. 8

2.1 RMSNorm vs LayerNorm & SwiGLU FFN Parameter Count Math (LLM05) ..... p. 9

2.2 MHA vs MQA vs GQA Parameter & KV-Cache Math (LLM05 , LLM06) ..... p. 11

2.3 RoPE (Rotary Position Embedding), ALiBi & Context Extension (LLM07) ..... p. 12

2.4 Sparse Mixture-of-Experts (MoE — Mixtral 8x7B / DeepSeek-MoE) ..... p. 14

2.5 Chinchilla Compute-Optimal Scaling Laws vs. Edge Inference Over-Training ..... p. 15

3. LLM Systems, Memory Arithmetic & Inference Optimization ..... p. 16

3.1 Prefill vs Decode Phases & The Hardware Roofline Model (LLM10) ..... p. 16

3.2 Exact KV-Cache Memory Calculation (LLM08) ..... p. 18

3.3 Mixed-Precision Training Memory (16Ψ Rule) & Activation Checkpointing (LLM09) ..... p. 19

3.4 FlashAttention (IO-Aware Tiling) & PagedAttention (vLLM) (LLM10) ..... p. 21

3.5 Decoding & Sampling Strategies (LLM11) ..... p. 23

4. Fine-Tuning (PEFT / LoRA) & Alignment (RLHF vs DPO vs GRPO) ..... p. 24

4.1 LoRA & QLoRA Math & Exam Traps (LLM12 , LLM13) ..... p. 24

4.2 Post-Training Alignment: SFT → RLHF (PPO) vs DPO vs GRPO (LLM14) ..... p. 26

5. Edge AI, Quantization, Distillation & Fast Inference (Qualcomm Focus) ..... p. 29

5.1 Floating-Point Formats: FP32 vs FP16 vs BF16 (LLM15) ..... p. 29

5.2 Exact Quantization Math: Asymmetric (Affine) vs Symmetric (LLM16 , LLM17) ..... p. 31

5.3 PTQ vs QAT & Modern LLM Quantization (GPTQ, AWQ, SmoothQuant) (LLM18) ..... p. 33

5.4 Knowledge Distillation, Pruning, Speculative Decoding & RAG (LLM19 , LLM20) ..... p. 34

6. Master Quick-Reference Checklist (LLM01 – LLM20) ..... p. 37

> ⚡ **IMPORTANT**
>
> **How to Read This Textbook:** If Transformers, Large Language Models (LLMs), and Efficient Edge AI aren't your daily bread and butter, dense research papers can feel like a wall of acronyms (`GQA` , `RoPE` , `SwiGLU` , `KV-Cache` , `FlashAttention` , `LoRA` , `DPO` , `W4A16`).
>
> For the **Qualcomm AI Research (QARE) Exam**, this is the flagship pillar. Every single subsection in this textbook is structured in **four clear steps**:
>
> 1. 💡 **Intuition (Plain English First):** What real-world bottleneck does this solve, and why did engineers invent it?
> 2. 📐 **The Math & Every Symbol Explained:** Every matrix shape is written using our standard $n \times m$ **(n rows, m columns)** convention, and every symbol is defined in plain English.
> 3. 🔢 **Step-by-Step Worked Toy Example:** Concrete numbers worked out line by line so you can trace every matrix multiplication and memory byte by hand.
> 4. 🎯 **Exam Takeaway & Common Traps:** The 60-second rule of thumb and exact traps tested in HackerRank questions `LLM01` – `LLM20` .

---

1. The Transformer Architecture from First Principles

↑ Mục Lục

1.1 Why Self-Attention Replaced Recurrence

💡 Intuition (Plain English First)

Before 2017, sequence models used Recurrent Neural Networks (RNNs and LSTMs). Imagine reading a 1,000-word book where you are forced to whisper a summary of everything you've read so far into a single sticky note (h_t) and pass it word-by-word from left to right:

1. **You can't read in parallel:** You cannot compute the sticky note for word 500 until you have finished computing words $1, 2, \ldots, 499$. Modern GPUs have thousands of cores sitting idle waiting for the previous word!
2. **You forget the beginning:** By the time the sticky note reaches word 500, details from word 1 have been overwritten or squashed by 499 matrix multiplications (vanishing gradients).

Self-Attention throws away the sequential sticky note. Instead, every token in the sentence looks directly at **every other token in the sentence at the exact same time**—like a room full of people where anyone can make direct eye contact with anyone else in 1 step!

📐 The Math & Every Symbol Explained

For a sequence of N tokens (where each token is represented by an embedding vector of size D), our input matrix X has $n = N$ rows and $m = D$ columns ($X \in \mathbb{R}^{N \times D}$):

Architecture	Sequential Steps (Training Parallelism)	Max Path Length Between Token 1 and Token N	Compute per Layer (N tokens, dim D)
RNN / LSTM	$\mathcal{O}(N)$ sequential steps (cannot parallelize across time)	$\mathcal{O}(N)$ steps (gradients vanish over long distance)	$\mathcal{O}(ND^2)$
Self-Attention	$\mathcal{O}(1)$ **steps** (all N tokens processed in parallel!)	$\mathcal{O}(1)$ **step** (direct token-to-token connection)	$\mathcal{O}(N^2D + ND^2)$

- N **(or** S**):** Sequence length (number of tokens in the prompt/sentence, i.e., number of rows in X).
- D **(or** d_{model}**):** Hidden dimension of the model (number of columns in X, e.g., 4096 in LLaMA-3-8B).

1.2 Scaled Dot-Product Attention & The $\sqrt{d_k}$ Variance Proof (`LLM01`)

💡 Intuition (Plain English First)

How does Self-Attention decide which words should pay attention to which other words? It uses a **database lookup analogy** with three vectors created from every token:

- **Query (q):** "*What am I looking for?*" (e.g., the pronoun `"it"` looking for the noun it refers to).
- **Key (k):** "*What label describes me?*" (e.g., the noun `"trophy"` advertising that it is an inanimate object).
- **Value (v):** "*What actual information do I contain if you pay attention to me?*"

To measure how well token i's Query matches token j's Key, we take their dot product $q_i \cdot k_j$. **But why do we divide that dot product by $\sqrt{d_k}$?** Because if two vectors have $d_k = 64$ or 128 random numbers in them, adding up 128 products creates a sum with a huge spread (large positive and negative swings like $+25$ or -20). When you feed numbers like $+25$ into softmax(z), e^{25} completely crushes everything else: one probability becomes 0.99999999 and all other probabilities become 0.00000000. In that flat region of Softmax, the gradient is zero (**vanishing gradients**), and the model stops learning! Dividing by $\sqrt{d_k}$ shrinks the spread back down to a healthy standard deviation of 1.

📐 The Math & Every Symbol Explained

Given an input sequence matrix $X \in \mathbb{R}^{N \times D}$ (N rows of tokens, D columns of features), we multiply by three learned weight matrices:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

- $X \in \mathbb{R}^{N \times D}$**:** Input token matrix (N token rows, D feature columns).
- $W_Q, W_K \in \mathbb{R}^{D \times d_k}$ **and** $W_V \in \mathbb{R}^{D \times d_v}$**:** Learned projection matrices mapping D input columns to d_k (query/key dimension) or d_v (value dimension) columns.
- $Q, K \in \mathbb{R}^{N \times d_k}$ **and** $V \in \mathbb{R}^{N \times d_v}$**:** The resulting Query, Key, and Value matrices (N rows, d_k or d_v columns).
- $QK^T \in \mathbb{R}^{N \times N}$**:** The pairwise similarity matrix (N query rows by N key columns), where entry (i, j) is the dot product $q_i \cdot k_j$.
- $\sqrt{d_k}$**:** Scaling factor (square root of the key dimension d_k).

- $M \in \mathbb{R}^{N \times N}$: Optional attention mask (0 for allowed positions, $-\infty$ for blocked future positions).

> 📌 **NOTE**
>
> **Step-by-Step Mathematical Proof of the $\sqrt{d_k}$ Scaling Factor (and the $\sigma^2 \neq 1$ Exam Twist):** Suppose the components q_i and k_i $(i = 1, \ldots, d_k)$ are independent random variables with mean $\mathbb{E}[q_i] = \mathbb{E}[k_i] = 0$ and variance $\mathrm{Var}(q_i) = \mathrm{Var}(k_i) = \sigma^2$:
>
> 1. **Mean of a single product $q_i k_i$:**
>
> $$\mathbb{E}[q_i k_i] = \mathbb{E}[q_i] \cdot \mathbb{E}[k_i] = 0 \times 0 = 0$$
>
> 2. **Variance of a single product $q_i k_i$:**
>
> $$\mathrm{Var}(q_i k_i) = \mathbb{E}[(q_i k_i)^2] - (\mathbb{E}[q_i k_i])^2 = \mathbb{E}[q_i^2] \cdot \mathbb{E}[k_i^2] - 0 = \sigma^2 \cdot \sigma^2 = \sigma^4$$
>
> 3. **Variance and Standard Deviation of the sum $s = q \cdot k = \sum_{i=1}^{d_k} q_i k_i$:** By independence, the variance of the sum of d_k terms is the sum of their variances:
>
> $$\mathrm{Var}(s) = \sum_{i=1}^{d_k} \sigma^4 = d_k \sigma^4 \implies \mathrm{Std}(s) = \sqrt{d_k \sigma^4} = \sigma^2 \sqrt{d_k}$$
>
> 4. **How to scale s to restore unit variance ($\mathrm{Var} = 1$):**
>
> - **Standard Case ($\sigma^2 = 1$):** $\mathrm{Var}(s) = d_k$, so dividing by $\sqrt{d_k}$ multiplies the variance by $\left(\frac{1}{\sqrt{d_k}}\right)^2 = \frac{1}{d_k}$, giving $\mathrm{Var}\left(\frac{s}{\sqrt{d_k}}\right) = 1$.
> - **Generalized Exam Twist (`LLM01`, $\sigma^2 \neq 1$):** If $\mathrm{Var}(q_i) = \mathrm{Var}(k_i) = \sigma^2$, you must divide by $\sigma^2 \sqrt{d_k}$ so that $\mathrm{Var}\left(\frac{s}{\sigma^2 \sqrt{d_k}}\right) = \frac{d_k \sigma^4}{\sigma^4 d_k} = 1$!

🔢 Step-by-Step Worked Toy Example

Suppose a tiny head has dimension $d_k = 64$, and the elements of q and k are drawn independently with mean 0 and variance $\sigma^2 = 2$:

1. **Step 1 — Variance of one element product $q_i k_i$:**

$$\mathrm{Var}(q_i k_i) = \sigma^2 \times \sigma^2 = 2 \times 2 = 4$$

2. **Step 2 — Variance of the 64-dimensional dot product $s = \sum_{i=1}^{64} q_i k_i$:**

$$\mathrm{Var}(s) = 64 \times 4 = 256$$

3. **Step 3 — Find the exact divisor c so that $\mathrm{Var}(s/c) = 1$:**

$$\frac{\mathrm{Var}(s)}{c^2} = 1 \implies c = \sqrt{256} = 16 = \sigma^2 \sqrt{d_k} = 2\sqrt{64}$$

(If you only divided by $\sqrt{d_k} = 8$, the variance would still be $256/64 = 4$, not 1!)

🎯 Exam Takeaway & Common Traps

- **Trap in `LLM01`:** When an exam question specifies $\text{Var}(q_i) = \text{Var}(k_i) = \sigma^2 = 2$, candidates who memorized $\sqrt{d_k}$ pick $\sqrt{2d_k}$ by mistake. Remember: two independent factors of variance σ^2 multiply to σ^4, whose square root is $\sigma^2\sqrt{d_k} = 2\sqrt{d_k}$!

1.3 Multi-Head Attention (MHA): Shapes, Parameters & The $N = 2D$ Crossover (`LLM02`, `LLM03`)

💡 Intuition (Plain English First)

If a sentence says "*The animal didn't cross the street because it was too tired*", one word needs to track **multiple types of relationships simultaneously**:

- Head 1 might track **pronoun coreference** (`"it"` → `"animal"`).
- Head 2 might track **verb-object grammar** (`"cross"` → `"street"`).
- Head 3 might track **causal conjunctions** (`"because"` → `"tired"`).

Instead of running one giant attention lookup of width $D = 4096$, **Multi-Head Attention (MHA)** chops the D-dimensional vector into h smaller slices (heads), each of width $d_k = D/h$ (e.g., $h = 32$ heads of width $d_k = 128$). Each head computes its own independent attention map, and then we glue (concatenate) the h slices back together into a vector of width D!

📐 The Math & Every Symbol Explained

$$\text{head}_i = \text{Attention}\left(XW_Q^{(i)},\ XW_K^{(i)},\ XW_V^{(i)}\right), \qquad \text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O$$

Let's trace the exact **PyTorch tensor shapes** through an MHA layer with Batch size B, Sequence length N rows, Model dimension D columns, h heads, and per-head dimension $d_k = D/h$:

1. **Input tensor X:** Shape (B, N, D) — B batches of $N \times D$ matrices.
2. **Combined Linear Projections (Q, K, V):** Multiply (B, N, D) by $W_Q, W_K, W_V \in \mathbb{R}^{D\times D}$ → shape (B, N, D).
3. **Split into h heads & transpose:** Reshape $(B, N, D) \to (B, N, h, d_k)$ and swap axes $1 \leftrightarrow 2$ → shape (B, h, N, d_k) (so each head is an $N \times d_k$ matrix).
4. **Attention Scores ($QK^T/\sqrt{d_k}$):** Multiply (B, h, N, d_k) by transposed keys (B, h, d_k, N) → shape (B, h, N, N).
5. **Weighted Values (Scores $\cdot\ V$):** Multiply (B, h, N, N) by (B, h, N, d_k) → shape (B, h, N, d_k).
6. **Concatenate heads & Output Projection W_O:** Swap axes back $(B, N, h, d_k) \to (B, N, D)$ and multiply by $W_O \in \mathbb{R}^{D\times D}$ → final shape (B, N, D).

- **Total Parameter Count of MHA (ignoring biases):**
  - All h heads of $W_Q^{(i)} \in \mathbb{R}^{D\times d_k}$ stacked side-by-side form a single $D \times D$ matrix $W_Q \in \mathbb{R}^{D\times D}$ (D^2 parameters).
  - Same for $W_K \in \mathbb{R}^{D\times D}$, $W_V \in \mathbb{R}^{D\times D}$, and $W_O \in \mathbb{R}^{D\times D}$.
  - **Total MHA Parameters: $\mathbf{4D^2}$** (or $4D^2 + 4D$ if biases are enabled).
- **Total Compute (MACs and FLOPs) of MHA for 1 Sequence of N Tokens:** Recall that multiplying an $n \times m$ matrix by an $m \times p$ matrix costs $n \cdot m \cdot p$ Multiply-Accumulates (MACs), which equals $2 \cdot n \cdot m \cdot p$ FLOPs:
  - **4 Linear Projections (Q, K, V, O):** Each multiplies $(N \times D)$ by $(D \times D) \implies 4 \times ND^2 = \mathbf{4ND^2}$ MACs ($8ND^2$ FLOPs).

- **2 Attention Matrix Multiplications (QK^T and PV across h heads):**
  - QK^T: h heads $\times (N \times d_k) \times (d_k \times N) = h \cdot N^2 d_k = N^2 D$ MACs ($2N^2 D$ FLOPs).
  - PV: h heads $\times (N \times N) \times (N \times d_k) = h \cdot N^2 d_k = N^2 D$ MACs ($2N^2 D$ FLOPs).
  - **Total Quadratic Attention Ops: $\mathbf{2N^2D}$** MACs ($4N^2 D$ FLOPs).

🔢 Step-by-Step Worked Toy Example

Let's solve two classic exam questions with $D = 512$:

1. **Question A (`LLM02`): Suppose we double the number of heads from $h = 8$ ($d_k = 64$) to $h = 16$ ($d_k = 32$) while keeping $D = 512$ fixed. How do the parameter count and attention matrix memory change?**
   - **Parameter count at $h = 8$:** $4D^2 = 4 \times 512^2 = 1,048,576$ parameters.
   - **Parameter count at $h = 16$:** Because $h \times d_k = 16 \times 32 = 512 = D$, the combined matrices W_Q, W_K, W_V, W_O are still 512×512! The parameter count changes by 0!
   - **Attention score matrix memory (B, h, N, N):** Doubles by $2\times$ because there are now 16 attention maps of size $N \times N$ instead of 8.
2. **Question B (`LLM03`): At what exact sequence length N do the quadratic attention matrix multiplications (QK^T and PV) equal the FLOPs of the 4 linear projections (W_Q, W_K, W_V, W_O)?**
   - Set Quadratic Attention FLOPs equal to Linear Projection FLOPs:

     $$4N^2 D = 8ND^2$$

   - Divide both sides by $4ND$:

     $$N = 2D$$

   - For $D = 512$, the crossover is at $N = 2 \times 512 = 1024$ **tokens** (and for LLaMA-3-8B with $D = 4096$, the crossover is at $N = 8192$ **tokens**!).

🎯 Exam Takeaway & Common Traps

- **The h-Invariance Rule (`LLM02`):** Changing the number of heads h while keeping $D = h \cdot d_k$ constant **never changes the parameter count ($4D^2$)** or the total FLOPs—it only changes the (B, h, N, N) activation memory!
- **The $N = 2D$ Crossover (`LLM03`):** For short prompts ($N < 2D$), linear projections ($8ND^2$ FLOPs) dominate MHA compute. Only when context length exceeds $N > 2D$ does quadratic attention ($4N^2 D$ FLOPs) become the dominant bottleneck!

1.4 Causal Masking, Pre-LN vs Post-LN & Encoder vs Decoder Paradigms (`LLM04`)

💡 Intuition (Plain English First)

- **Causal Masking:** When training an autoregressive LLM (like GPT or LLaMA) to predict the next word, all N words of the training sentence are fed in at once for parallel speed. If word 3 is allowed to peek at word 4

during self-attention, the model will just copy word 4 with 100% accuracy and learn nothing! To prevent cheating, we put a blindfold (**Causal Mask**) over all future positions $j > i$.

  - *Why must the blindfold be $-\infty$ instead of 0?* Because the attention scores go into softmax$(s_j) = \frac{e^{s_j}}{\sum_k e^{s_k}}$. If you set a future score to 0, then $e^0 = 1$, which gets a positive probability! You must set blocked positions to $-\infty$ before Softmax so that $e^{-\infty} = 0$.

- **Pre-LN vs Post-LN:** In the original 2017 Transformer (**Post-LN**), LayerNorm sat *on the main highway* after the residual addition: $x_{l+1} = \text{LN}(x_l + F(x_l))$. During backpropagation through $L = 32$ layers, gradients get divided by the LayerNorm variance at every single floor, causing gradients at the bottom layers to vanish while gradients at the top layers explode. Modern LLMs use **Pre-LN**: $x_{l+1} = x_l + F(\text{LN}(x_l))$. Now the main residual highway $x_L = x_0 + \sum_{l=1}^{L} F_l(\text{LN}(x_{l-1}))$ is a clean, unobstructed identity wire from the loss all the way back to layer 0!

📐 The Math & Every Symbol Explained

1. **Causal Attention Mask Matrix $M \in \mathbb{R}^{N \times N}$ (N query rows, N key columns):**

$$M_{ij} = \begin{cases} 0 & \text{if } j \le i \quad \text{(past and present tokens allowed)} \\ -\infty & \text{if } j > i \quad \text{(future tokens blocked: } \exp(-\infty) = 0\text{)} \end{cases}$$

2. **Post-LN vs Pre-LN Residual Equations:**
   - **Post-LN (2017 Vaswani Transformer):** $x_{l+1} = \text{LN}\big(x_l + \text{Sublayer}(x_l)\big)$
   - **Pre-LN (GPT, LLaMA, Qwen, ViT):** $x_{l+1} = x_l + \text{Sublayer}\big(\text{LN}(x_l)\big)$

3. **Transformer Paradigm Taxonomy:**

Paradigm	Attention Mask	Pretraining Objective	Representative Models	Best Use Cases
Encoder-Only	Bidirectional (No mask; all $N \times N$ entries active)	**Masked Language Modeling (MLM):** mask 15% of tokens (80% `[MASK]`, 10% random, 10% unchanged)	BERT, RoBERTa, DeBERTa	Dense embeddings, text classification, NER, Cross-Encoder reranking
Decoder-Only	Causal lower-triangular ($M_{ij} = -\infty$ for $j > i$)	**Next-Token Prediction (Causal LM):** $\sum_t \log P(x_t \mid x_{<t})$	GPT-4, LLaMA-3, Qwen, Mistral	Text generation, reasoning, chat, code
Encoder-Decoder	Bidirectional Encoder + Cross-Attention + Causal Decoder	Span corruption / Seq2Seq denoising	T5, BART, Whisper	Translation, speech-to-text (Whisper)

🔢 Step-by-Step Worked Toy Example

Suppose we have a 3-token sequence (`"I"`, `"love"`, `"AI"`) and we are computing the attention probabilities for **Row** $i = 1$ (the first token `"I"`, which is only allowed to look at itself, $j = 1$, and must NOT look at $j = 2$ or $j = 3$). Suppose the raw scaled dot products $\frac{q_1 \cdot k_j}{\sqrt{d_k}}$ for row 1 are $[2.0,\ 5.0,\ 5.0]$:

- **Buggy Implementation (setting future scores to 0 before Softmax):**
  - Masked scores: $[2.0,\ 0.0,\ 0.0]$.
  - Softmax: $\left[\frac{e^2}{e^2+e^0+e^0}, \frac{e^0}{e^2+2}, \frac{e^0}{e^2+2}\right] = \left[\frac{7.39}{9.39}, \frac{1}{9.39}, \frac{1}{9.39}\right] = [0.787,\ \mathbf{0.106},\ \mathbf{0.106}]$.

- **Disaster:** Over 21% of token 1's output is leaked from future tokens 2 and 3!

- **Correct Implementation (adding $M_{1,2} = M_{1,3} = -\infty$ before Softmax):**
  - Masked scores: $[2.0,\ -\infty,\ -\infty]$.
  - Softmax: $\left[\frac{e^2}{e^2+0+0}, \frac{0}{e^2}, \frac{0}{e^2}\right] = [\mathbf{1.0},\ \mathbf{0.0},\ \mathbf{0.0}]$. Zero future leakage!

🎯 Exam Takeaway & Common Traps

- **Causal Masking (`LLM04`):** Always add $-\infty$ (or `-1e9` in FP16) **before** Softmax, never 0!
- **Why BERT MLM uses the 80-10-10 rule:** During fine-tuning/inference, the special `[MASK]` token never appears. Keeping 10% random tokens and 10% original tokens forces BERT to maintain high-quality contextual representations for *every* token, not just `[MASK]` tokens.

---

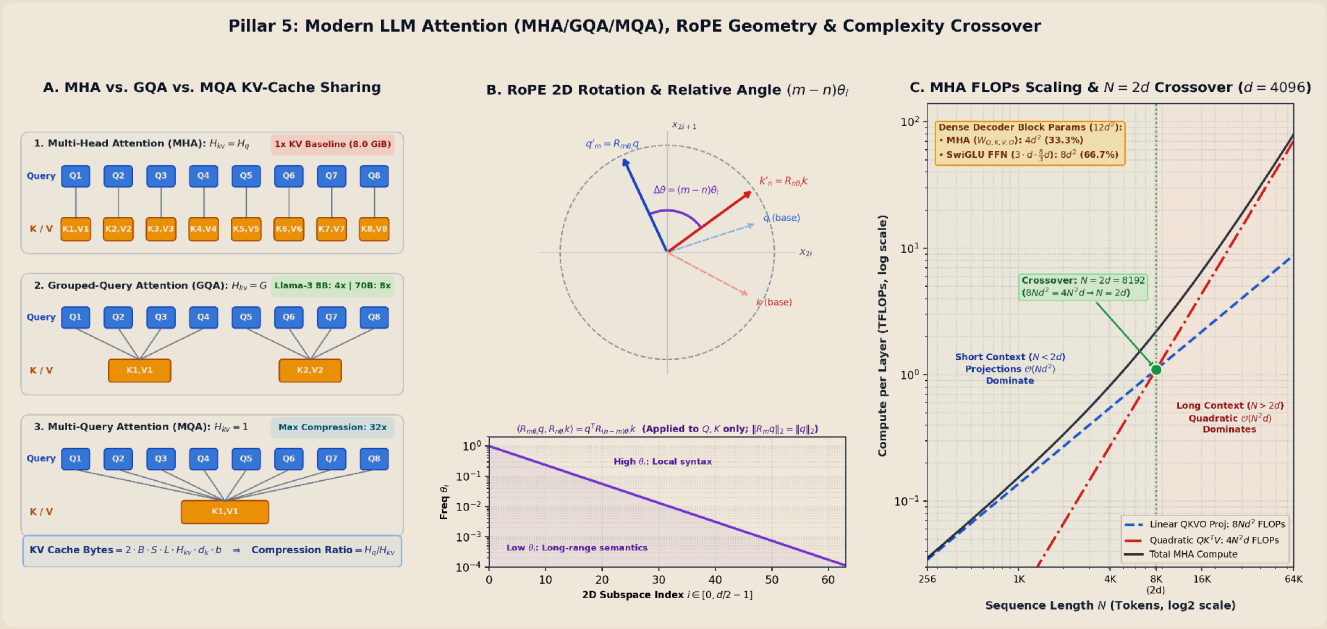
2. Modern LLM Architectural Upgrades (LLaMA / Qwen / Mistral DNA)

↑ Mục Lục

Modern frontier LLMs (LLaMA-3, Qwen-2.5, Mistral, DeepSeek) replace **five components** of the original 2017 Transformer. Know this comparison table cold:

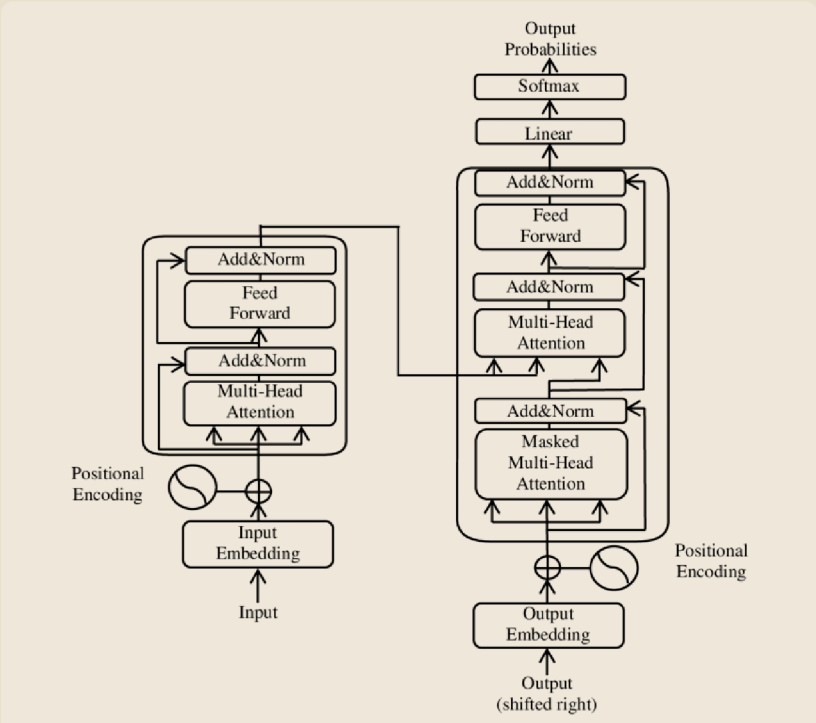
Component	Original 2017 Transformer	Modern LLM (LLaMA-3 / Qwen / Mistral)	Mathematical & Systems Advantage
Normalization	Post-LayerNorm ($2D$ params/layer: γ, β)	**Pre-RMSNorm** (D params/layer: γ only)	Drops mean-centering (μ) and bias (β); saves $\sim$ 7–15% latency with equal accuracy.
Activation / FFN	$\text{ReLU}(xW_1)W_2$ (2 matrices, $D_{\text{ff}} = 4D$)	**SwiGLU** (3 matrices: $W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$)	Gated linear unit $(\text{Swish}(xW_g) \odot xW_u)W_d$ yields consistently lower perplexity.
Positional Encoding	Sinusoidal / Learned Absolute added to input	**RoPE (Rotary Position Embedding)**	Applied to Q, K at every layer; inner product depends strictly on relative distance $m - n$.
Attention Heads	Multi-Head Attention (MHA: $h_{kv} = h_q$)	**Grouped-Query Attention (GQA: $h_{kv} < h_q$)**	Shares K, V heads across groups of Q heads; slashes KV-Cache memory by h_q/h_{kv} (4–8×).
Linear Biases	`bias=True` in Linear & LayerNorm	**`bias=False`** everywhere	Reduces parameter count and improves FP16/BF16 numerical stability at scale.

FIGURE SLIDE 1 OF 2



Modern LLM Attention (MHA/GQA/MQA), RoPE 2D Rotation & Complexity Crossover

FIGURE SLIDE 2 OF 2



Canonical 2017 Transformer Architecture (Vaswani et al.)

2.1 RMSNorm vs LayerNorm & SwiGLU FFN Parameter Count Math (`LLM05`)

💡 Intuition (Plain English First)

1. **Why RMSNorm replaced LayerNorm:** Standard LayerNorm does two passes over every token vector $x \in \mathbb{R}^D$: first it computes the mean μ to center the vector $(x - \mu)$, and second it computes the variance σ^2 to scale the vector. Zhang & Sennrich (2019) discovered that **re-centering the mean to zero does nothing for accuracy**—only scaling the vector's magnitude matters! **RMSNorm** skips the mean μ and shift β completely, dividing directly by the Root Mean Square (RMS) of the vector. That cuts memory reads/writes inside the normalization kernel and cuts norm parameters in half (D instead of $2D$).
2. **Why SwiGLU replaced ReLU FFN:** In a standard Feed-Forward Network (FFN), a single matrix W_1 projects x up to $4D$, applies ReLU, and W_2 projects back down to D. In **SwiGLU**, we use **two parallel up-projections**: one acts as a "content stream" (xW_{up}) and the other acts as a smooth, learnable "valve/gate" ($\text{Swish}(xW_{\text{gate}})$) that controls how much of each feature passes through via element-wise multiplication ($\odot$), before W_{down} projects back to D. Because SwiGLU uses **3 matrices instead of 2**, we shrink the intermediate width from $4D$ to $\frac{8}{3}D \approx 2.67D$ so total parameters stay the same ($3 \times D \times \frac{8}{3}D = 8D^2$).

📐 The Math & Every Symbol Explained

- **LayerNorm ($2D$ parameters: scale $\gamma \in \mathbb{R}^D$, shift $\beta \in \mathbb{R}^D$):**

$$\text{LN}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta, \qquad \text{where } \mu = \frac{1}{D}\sum_{i=1}^{D} x_i, \quad \sigma^2 = \frac{1}{D}\sum_{i=1}^{D}(x_i - \mu)^2$$

- **RMSNorm (D parameters: scale $\gamma \in \mathbb{R}^D$ only, no μ, no β):**

$$\text{RMSNorm}(x) = \frac{x}{\text{RMS}(x)} \odot \gamma, \qquad \text{where RMS}(x) = \sqrt{\frac{1}{D}\sum_{i=1}^{D} x_i^2 + \epsilon}$$

- **SwiGLU Feed-Forward Network ($3 \cdot D \cdot D_{\text{ff}}$ parameters with `bias=False`):**

$$\text{SwiGLU}(x) = \Big(\text{Swish}_1(xW_{\text{gate}}) \odot (xW_{\text{up}})\Big)W_{\text{down}}, \qquad \text{where Swish}_1(z) = z \cdot \sigma(z) = \frac{z}{1 + e^{-z}}$$

  - $x \in \mathbb{R}^{N \times D}$: Input token matrix ($N$ rows, D columns).
  - $W_{\text{gate}} \in \mathbb{R}^{D \times D_{\text{ff}}}$: Gate projection matrix (D rows, D_{ff} columns).
  - $W_{\text{up}} \in \mathbb{R}^{D \times D_{\text{ff}}}$: Up-projection matrix ($D$ rows, D_{ff} columns).
  - $W_{\text{down}} \in \mathbb{R}^{D_{\text{ff}} \times D}$: Down-projection matrix (D_{ff} rows, D columns).
  - $\odot$: Element-wise (Hadamard) multiplication of the two $N \times D_{\text{ff}}$ matrices.

🔢 Step-by-Step Worked Toy Example

Let's normalize a tiny 2-element vector $x = [3.0,\ 4.0]$ with $\epsilon = 0$ and $\gamma = [1.0,\ 1.0]$ using **RMSNorm**:

1. **Step 1 — Compute the mean of squares:**

$$\frac{1}{2}(3.0^2 + 4.0^2) = \frac{9 + 16}{2} = \frac{25}{2} = 12.5$$

2. **Step 2 — Take the square root to get $\text{RMS}(x)$:**

$$\text{RMS}(x) = \sqrt{12.5} \approx 3.5355$$

3. **Step 3 — Divide x by $\text{RMS}(x)$:**

$$\text{RMSNorm}(x) = \left[\frac{3.0}{3.5355}, \frac{4.0}{3.5355}\right] \approx [0.8485,\ 1.1314]$$

(Notice that the mean of the output is $\approx 0.99 \neq 0$, which is completely fine—its root-mean-square is now exactly 1.0!)

2.2 MHA vs MQA vs GQA Parameter & KV-Cache Math (`LLM05`, `LLM06`)

💡 Intuition (Plain English First)

During LLM text generation, the biggest bottleneck is **reading past Keys and Values (the KV-Cache) from memory** for every single new word generated.

- In standard **Multi-Head Attention (MHA)**, if you have $h_q = 32$ Query heads, you also store 32 separate Key heads and 32 separate Value heads in memory.
- **Multi-Query Attention (MQA)** went to the extreme: all 32 Query heads share **just 1 single Key head and 1 single Value head** ($h_{kv} = 1$). That slashes KV-Cache memory by $32\times$, but hurts model accuracy slightly.
- **Grouped-Query Attention (GQA)** is the goldilocks sweet spot used in LLaMA-3, Qwen, and Mistral: we divide the $h_q = 32$ Query heads into $g = 8$ groups (so 4 Query heads per group). Each group shares 1 Key head and 1 Value head ($h_{kv} = 8$). We get $4\times$ **smaller KV-Cache** with virtually **zero loss in accuracy**!

📐 The Math & Every Symbol Explained

For model width D, h_q Query heads of width $d_k = D/h_q$, and h_{kv} Key/Value heads of the same width d_k:

- $W_Q \in \mathbb{R}^{D \times (h_q \cdot d_k)} = \mathbb{R}^{D \times D}$: Always has D^2 parameters.
- $W_O \in \mathbb{R}^{(h_q \cdot d_k) \times D} = \mathbb{R}^{D \times D}$: Always has D^2 parameters.
- $W_K, W_V \in \mathbb{R}^{D \times (h_{kv} \cdot d_k)}$: Each has $D \cdot h_{kv} \cdot d_k = D^2 \left(\frac{h_{kv}}{h_q}\right)$ parameters!
- **Total GQA Attention Parameter Formula (`bias=False`):**

$$\text{Params}_{\text{GQA}} = \underbrace{2D^2}_{W_Q, W_O} + \underbrace{2 \cdot D \cdot h_{kv} \cdot d_k}_{W_K, W_V} = 2D^2 \left(1 + \frac{h_{kv}}{h_q}\right)$$

- **KV-Cache Reduction Factor vs MHA:**

$$\text{KV-Cache Savings Ratio} = \frac{h_q}{h_{kv}}$$

🔢 Step-by-Step Worked Toy Example (`LLM05`, `LLM06`)

Let's calculate the **exact parameter count of 1 full LLaMA-style Decoder Layer** (`LLM05`) with:

- Model dimension $D = 4096$
- Query heads $h_q = 32 \implies$ head dimension $d_k = 4096/32 = 128$
- GQA Key/Value heads $h_{kv} = 8$ (so $32/8 = 4\times$ KV-Cache reduction)
- SwiGLU intermediate dimension $D_{\text{ff}} = 11008$

- 2 RMSNorm layers per block (one before Attention, one before FFN), `bias=False` everywhere:

1. **Step 1 — GQA Self-Attention Parameters:**
   - W_Q (4096×4096) and W_O (4096×4096):

     $$2 \times (4096 \times 4096) = 2 \times 16,777,216 = 33,554,432$$

   - W_K $(4096 \times (8 \times 128) = 4096 \times 1024)$ and W_V (4096×1024):

     $$2 \times (4096 \times 1024) = 2 \times 4,194,304 = 8,388,608$$

   - **Subtotal (GQA Attention):** $33,554,432 + 8,388,608 = \mathbf{41,943,040}$
2. **Step 2 — SwiGLU FFN Parameters (3 matrices: $W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$):**

   $$3 \times (4096 \times 11008) = 3 \times 45,088,768 = \mathbf{135,266,304}$$

3. **Step 3 — Two Pre-RMSNorm Layers (each has 1 vector $\gamma \in \mathbb{R}^{4096}$):**

   $$2 \times 4096 = \mathbf{8,192}$$

4. **Step 4 — Grand Total for 1 Decoder Layer:**

   $$41,943,040 + 135,266,304 + 8,192 = \mathbf{177,217,536} \text{ parameters } (\approx 177.22\text{M})$$

🎯 Exam Takeaway & Common Traps

- **3 Traps in `LLM05`:**
  1. Don't use $4D^2$ for GQA attention—W_K and W_V are shrunk by h_{kv}/h_q!
  2. Don't use $2 \cdot D \cdot D_{\text{ff}}$ for SwiGLU—SwiGLU has **3 matrices** ($W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$)!
  3. Don't use $2D$ per RMSNorm—RMSNorm has no β shift, so each norm has only D parameters ($2D$ total for the two RMSNorms in a layer)!

2.3 RoPE (Rotary Position Embedding), ALiBi & Context Extension (`LLM07`)

💡 Intuition (Plain English First)

Self-attention is completely permutation-invariant: without position markers, `"The dog bit the man"` and `"The man bit the dog"` look identical to the attention matrix.

- **Old way (Absolute Positional Encoding):** Add a fixed vector p_m to word m at the very bottom of the network. Problem: if the model was trained on positions $1 \ldots 2048$, it has no idea what position 3000 means, and it struggles to know that position 5 and position 7 have the exact same relative gap $(+2)$ as position 1005 and position 1007.
- **Modern way (RoPE — Rotary Position Embedding):** Instead of adding a number to the vector, imagine each pair of numbers (q_{2i}, q_{2i+1}) in a Query or Key vector as the hand of a clock on a 2D dial.
  - If a word is at position m, we rotate its clock hand by $m \cdot \theta$ degrees.
  - If another word is at position n, we rotate its clock hand by $n \cdot \theta$ degrees.

- When we take the dot product between the two rotated clock hands ($q_m \cdot k_n$), the dot product of two 2D vectors depends **only on the angle between them**: $(m-n)\theta$! Thus, attention automatically knows the **exact relative distance** $m-n$ between any two tokens!

📐 The Math & Every Symbol Explained

RoPE groups the d_k coordinates of a Query vector q_m (at token position m) or Key vector k_n (at token position n) into $d_k/2$ two-dimensional pairs $(2i, 2i+1)$, and multiplies each pair by a 2×2 rotation matrix $R(m\theta_i)$:

$$\begin{pmatrix} q'_{2i} \\ q'_{2i+1} \end{pmatrix} = \underbrace{\begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix}}_{R(m\theta_i) \in \mathbb{R}^{2\times 2}} \begin{pmatrix} q_{2i} \\ q_{2i+1} \end{pmatrix}, \qquad \text{where } \theta_i = \text{base}^{-2i/d_k}$$

- $m, n \in \{0, 1, \ldots, N-1\}$**:** Token position indices in the sequence.
- $\theta_i = \text{base}^{-2i/d_k}$**:** Rotation frequency for coordinate pair i (fast rotation for small i, slow rotation for large i; base $= 10,000$ in LLaMA-1/2 or $500,000$ in LLaMA-3).
- **Why the Inner Product Depends Only on Relative Distance** $(m-n)$**:** Because 2×2 rotation matrices are orthogonal ($R(\alpha)^T = R(-\alpha)$ and $R(\alpha)R(\beta) = R(\alpha+\beta)$):

  $$\langle R(m\theta)q,\ R(n\theta)k \rangle = q^T R(m\theta)^T R(n\theta) k = q^T R\big((n-m)\theta\big) k$$

- **Norm Preservation:** Because $R(m\theta)$ is an orthogonal matrix, rotating a vector never changes its length: $\|R(m\theta)q\|_2 = \|q\|_2$.

🔢 Step-by-Step Worked Toy Example

Suppose a 2D query vector $q = [1, 0]^T$ sits at position $m = 5$ and an identical key vector $k = [1, 0]^T$ sits at position $n = 7$, with rotation step $\theta = 30^\circ$:

1. **Step 1 — Rotate** q **at position** $m = 5$ **(**$5 \times 30^\circ = 150^\circ$**):**

   $$q'_5 = R(150^\circ) \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \cos 150^\circ \\ \sin 150^\circ \end{pmatrix}$$

2. **Step 2 — Rotate** k **at position** $n = 7$ **(**$7 \times 30^\circ = 210^\circ$**):**

   $$k'_7 = R(210^\circ) \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \cos 210^\circ \\ \sin 210^\circ \end{pmatrix}$$

3. **Step 3 — Compute their dot product** $q'_5 \cdot k'_7$**:** Using the trigonometric identity $\cos\alpha\cos\beta + \sin\alpha\sin\beta = \cos(\beta - \alpha)$:

   $$q'_5 \cdot k'_7 = \cos(150^\circ)\cos(210^\circ) + \sin(150^\circ)\sin(210^\circ) = \cos(210^\circ - 150^\circ) = \cos(60^\circ) = \mathbf{0.5}$$

4. **Step 4 — Check positions** $m = 1005$ **and** $n = 1007$ **(same gap** $n - m = 2$**):**

   $$q'_{1005} \cdot k'_{1007} = \cos\big((1007 - 1005) \times 30^\circ\big) = \cos(60^\circ) = \mathbf{0.5}$$

   The attention score is **100% identical** regardless of absolute position!

▼ 📌 **Optional / Stage-2 Interview Bonus (Skip for Timed MCQ): Long-Context RoPE Extension (PI, NTK, YaRN) & ALiBi**

- **Position Interpolation (PI):** Linearly downscales position index $m \to m\frac{L_{\text{train}}}{L_{\text{target}}}$ so angles stay within the trained range, though it crowds high-frequency local distinctions.
- **NTK-Aware Scaling / YaRN:** Scales the RoPE base frequency ($\text{base}' = \text{base} \cdot \alpha^{d/(d-2)}$) or interpolates only low-frequency dimensions while leaving high-frequency local dimensions untouched.
- **ALiBi (Attention with Linear Biases):** Used in MPT/Falcon; adds a static, non-learned linear distance penalty directly to attention logits before softmax: $\text{softmax}\left(\frac{q_i^T k_j}{\sqrt{d_k}} - m_h \cdot |i-j|\right)$, where each head h gets a geometric slope $m_h = 2^{-8h/H}$.

🎯 Exam Takeaway & Common Traps

- **3 Guaranteed RoPE Facts (`LLM07`):**
  1. RoPE encodes **relative position** $(m-n)$ by applying an orthogonal rotation matrix to **absolute positions** m **and** n independently.
  2. RoPE is applied **only to Queries (Q) and Keys (K)** at every attention layer—**never to Values (V)**!
  3. Because rotation matrices are orthogonal, RoPE strictly preserves the L_2 norm of Q and K.

2.4 Sparse Mixture-of-Experts (MoE — Mixtral 8x7B / DeepSeek-MoE)

💡 Intuition (Plain English First)

In a dense 47B-parameter LLM, every single token—even a trivial word like `"the"`—has to multiply through all 47 billion parameters. That makes inference slow and expensive. **Sparse Mixture-of-Experts (MoE)** replaces each dense FFN layer with a team of E specialist FFN "Experts" (e.g., $E = 8$ experts in Mixtral 8x7B: one might specialize in Python code, one in math, one in French grammar). When a token arrives, a tiny traffic cop called a **Router** looks at the token and sends it to **only the top $k = 2$ experts**!

- **The Result:** You still need enough RAM/VRAM to hold all $E = 8$ experts ($\sim$ 47B **total parameters**), but for any single token, your GPU/NPU only does the math for $k = 2$ experts ($\sim$ 13B **active parameters per token**)!

📐 The Math & Every Symbol Explained

Given a token representation $x \in \mathbb{R}^D$ entering an MoE FFN layer with E experts $\{E_1, \ldots, E_E\}$ and router matrix $W_g \in \mathbb{R}^{D \times E}$:

$$G(x) = \text{Softmax}\Big(\text{TopK}(xW_g,\ k)\Big), \qquad y = \sum_{i \in \text{TopK}} G(x)_i\, E_i(x)$$

- $W_g \in \mathbb{R}^{D \times E}$**:** Lightweight linear router matrix (D rows, E expert columns) that scores how well each expert matches token x.
- $\text{TopK}(\cdot, k)$**:** Keeps the k highest router logits and sets the remaining $E - k$ logits to $-\infty$ before Softmax.
- $G(x)_i$**:** Routing weight (probability) assigned to selected expert i.

- $E_i(x)$**:** Output of the i-th SwiGLU FFN expert.
- **Router Collapse & Load-Balancing Auxiliary Loss:** During early training, whichever expert happens to get slightly better gradients gets picked more often by the router, trains even faster, and soon receives 100% of all tokens (**Router Collapse**), leaving the other $E-1$ experts dead! To prevent this, we add an **auxiliary load-balancing loss** $\mathcal{L}_{\text{aux}} = \alpha E \sum_{i=1}^{E} f_i P_i$ that penalizes uneven routing across a batch.

🔢 Step-by-Step Worked Toy Example

Suppose an MoE model has $E = 8$ experts per layer, where each single expert FFN has 5B parameters across all layers, and the shared Attention + Embeddings have 7B parameters. We use Top-$k = 2$ routing:

1. **Step 1 — Total Parameters (VRAM Footprint):**

   $$\text{Total Params} = \text{Shared } (7\text{B}) + 8 \times \text{Expert } (5\text{B}) = \mathbf{47}\text{B parameters}$$

   (You must load all 47B *parameters into RAM/VRAM because any expert could be called!)*

2. **Step 2 — Active Parameters per Token (Compute FLOPs):**

   $$\text{Active Params} = \text{Shared } (7\text{B}) + 2 \times \text{Expert } (5\text{B}) = \mathbf{17}\text{B parameters per token}$$

   (Each token only executes 2 *experts, giving the compute speed of a* 17B *model with the knowledge capacity of a* 47B *model!)*

🎯 Exam Takeaway & Common Traps

- **MoE Trade-off:** MoE slashes **FLOPs per token** (proportional to k active experts), NOT **VRAM footprint** (proportional to all E total experts). For batch size $B = 1$ edge inference, MoE is still memory-capacity heavy, whereas for large batches it gives massive throughput gains.

2.5 Chinchilla Compute-Optimal Scaling Laws vs. Edge Inference Over-Training

💡 Intuition (Plain English First)

Suppose your company gives you a fixed supercomputer budget of $C = 10^{23}$ FLOPs to pretrain an LLM. Should you train a **huge 70B model on a small dataset**, or a **smaller 10B model on** $7\times$ **more data**?

- In 2022, DeepMind's **Chinchilla paper** proved that to get the lowest loss for a fixed *training* budget C, you should scale model parameters N and training tokens D equally: **train for** ≈ 20 **tokens per parameter** ($D_{\text{opt}} \approx 20N$).
- **Wait—why did Meta train LLaMA-3-8B on** 15 Trillion **tokens (**$\approx 1,875$ **tokens per parameter, nearly** $100\times$ **Chinchilla)?!** Because Chinchilla only cared about **training cost** and ignored **inference cost**! If you want an LLM to run on a Qualcomm Snapdragon phone (where RAM and memory bandwidth are tight), you cannot fit a 70B model—you *must* ship a small 3B or 8B model. Spending extra supercomputer electricity once during training to "over-train" an 8B model on 15T tokens makes every single user query faster and cheaper forever!

📐 The Math & Every Symbol Explained

1. **Dense Transformer Training Compute Rule:**

$$C \approx 6 \cdot N \cdot D \quad \text{(in FLOPs)}$$

   - N **(or** Ψ**):** Number of non-embedding model parameters.
   - D**:** Number of training tokens processed.
   - **Where does the factor of** 6 **come from?**
     - Forward pass: 1 matrix multiply per weight per token $=$ 1 MAC $=$ **2** FLOPs.
     - Backward pass: computing gradients w.r.t. input activations costs 2 FLOPs, and computing gradients w.r.t. weights costs 2 FLOPs, totaling **4** FLOPs (twice the forward pass!).
     - Total per parameter per token: $2 + 4 =$ **6** FLOPs.

2. **Chinchilla Compute-Optimal Token Rule:**

$$D_{\text{opt}} \approx 20N \implies C_{\text{opt}} \approx 6 \cdot N \cdot (20N) = 120N^2 \text{ FLOPs}$$

🔢 Step-by-Step Worked Toy Example

Let's compare a Chinchilla-optimal 8B model vs. LLaMA-3-8B:

1. **Chinchilla-Optimal Training Tokens for** $N = 8\text{B}$**:**

$$D_{\text{opt}} = 20 \times 8\text{B} = \textbf{160} \text{ Billion tokens}$$

   - Training FLOPs: $C \approx 6 \times (8 \times 10^9) \times (1.6 \times 10^{11}) = 7.68 \times 10^{21}$ FLOPs.

2. **LLaMA-3-8B Over-Trained Setup (**$D = 15$ Trillion tokens**):**

$$\text{Tokens per Parameter} = \frac{15,000\text{B}}{8\text{B}} = \textbf{1,875} \text{ tokens/parameter } (\approx 94 \times \text{ Chinchilla!})$$

   - Training FLOPs: $C \approx 6 \times (8 \times 10^9) \times (1.5 \times 10^{13}) = 7.2 \times 10^{23}$ FLOPs.

🎯 Exam Takeaway & Common Traps

- **Memorize both numbers:**
  1. Training FLOPs $=$ **6ND** ($2ND$ forward $+4ND$ backward).
  2. Chinchilla compute-optimal tokens $=$ **20N** (violated intentionally by modern small/edge LLMs to minimize *inference* latency and memory footprint!).

---

3. LLM Systems, Memory Arithmetic & Inference Optimization

↑ Mục Lục

3.1 Prefill vs Decode Phases & The Hardware Roofline Model (`LLM10`)

💡 Intuition (Plain English First)

Think of a GPU or Qualcomm Hexagon NPU as a factory with two limits:

1. **How fast the workers can calculate inside the factory:** Peak Compute Π_{peak} (measured in FLOPs/sec).
2. **How fast the delivery trucks can haul weights from the warehouse (DRAM/HBM) to the factory:** Memory Bandwidth β_{BW} (measured in Bytes/sec).

On modern chips, the factory workers are $\sim 200\times$ **faster** than the delivery trucks! This means if you haul a weight matrix from DRAM and only use it once (for 1 token), the workers finish instantly and spend 99.5% of their time waiting on the delivery trucks (**Memory-Bandwidth-Bound**). This explains why LLM inference has **two completely different personalities**:

- **Phase 1: Prefill (Reading your 1,000-word prompt):** All $1,000$ prompt tokens are already available! We haul each weight matrix from DRAM **once** and multiply it against all $1,000$ tokens at the same time (**Matrix-Matrix Multiplication / GEMM**). The workers stay busy doing $1,000$ operations per byte loaded $\rightarrow$ **Compute-Bound**!
- **Phase 2: Decode (Generating the reply 1 word at a time):** Because word $t+1$ depends on word t, we can only generate 1 **token at a time**. For every single word generated, we have to haul the *entire* multi-gigabyte LLM weight matrix from DRAM just to multiply it by 1 single vector (**Matrix-Vector Multiplication / GEMV**)! We only do 1 operation per byte loaded $\rightarrow$ **Memory-Bandwidth-Bound**!

📐 The Math & Every Symbol Explained

1. **Operational Intensity (OI, also called Arithmetic Intensity):**

$$\text{OI} = \frac{\text{Total FLOPs Executed}}{\text{Total Bytes Read/Written from Memory}} \quad \left[\frac{\text{FLOPs}}{\text{Byte}}\right]$$

2. **The Roofline Throughput Equation & Ridge Point I_{ridge}:**

$$\text{Attainable Throughput (FLOPs/s)} = \min\Big(\underbrace{\Pi_{\text{peak}}}_{\text{Compute Ceiling}}, \underbrace{\text{OI} \times \beta_{\text{BW}}}_{\text{Memory Bandwidth Slope}}\Big), \qquad I_{\text{ridge}} = \frac{\Pi_{\text{peak}}}{\beta_{\text{BW}}}$$

   - **If** $\text{OI} < I_{\text{ridge}}$**:** Workload is **Memory-Bandwidth-Bound** (speed up by quantizing weights to INT4/INT8, using GQA, FlashAttention, or Speculative Decoding).
   - **If** $\text{OI} > I_{\text{ridge}}$**:** Workload is **Compute-Bound** (speed up by faster Tensor Cores or lower-precision MAC units).

3. **Operational Intensity of a Linear Layer (Ψ parameters stored in FP16 $= 2\Psi$ bytes) for $M = B \cdot S$ tokens:**

$$\text{OI} \approx \frac{2 \cdot M \cdot \Psi \text{ FLOPs}}{2\Psi \text{ Bytes}} = M = B \cdot S \text{ FLOPs/Byte}$$

   - **Prefill ($B = 1, S = 2048$):** $\text{OI} \approx 2048$ FLOPs/Byte $\gg I_{\text{ridge}} \implies$ **Compute-Bound**.
   - **Decode ($B = 1, S = 1$):** $\text{OI} \approx 1$ FLOP/Byte $\ll I_{\text{ridge}} \implies$ **Memory-Bandwidth-Bound**!

🔢 Step-by-Step Worked Toy Example

Suppose an edge NPU has Peak Compute $\Pi_{\text{peak}} = 100$ TFLOPs/s (10^{14} FLOPs/s) and LPDDR5X Memory Bandwidth $\beta_{\text{BW}} = 100$ GB/s (10^{11} Bytes/s), and we run a $\Psi = 7$B model in FP16 (14 GB of weights) at batch size $B = 1$:

1. **Step 1 — Find the Hardware Ridge Point I_{ridge}:**

$$I_{\text{ridge}} = \frac{100 \times 10^{12}\ \text{FLOPs/s}}{100 \times 10^{9}\ \text{Bytes/s}} = \mathbf{1000}\ \text{FLOPs/Byte}$$

2. **Step 2 — Calculate maximum Decode speed ($S = 1, \text{OI} = 1\ \text{FLOP/Byte} \ll 1000$):** Every generated token must read all 14 GB of weights over a 100 GB/s bus:

$$\text{Max Decode Speed} \approx \frac{\beta_{\text{BW}}}{\text{Model Weight Bytes}} = \frac{100\ \text{GB/s}}{14\ \text{GB/token}} \approx \mathbf{7.14}\ \text{tokens/sec}$$

3. **Step 3 — What happens if we quantize the weights from FP16 (14 GB) to INT4 (3.5 GB)?** Because Decode is memory-bandwidth-bound, shrinking the weight traffic by 4× speeds up token generation by 4× to $\frac{100}{3.5} \approx \mathbf{28.6}$ tokens/sec!

🎯 Exam Takeaway & Common Traps

- **Why do we use a KV-Cache?** During Decode step t, the query is only the newest token q_t $(1 \times d_k)$, but it must attend to all past keys $K_{1:t}$ and values $V_{1:t}$. Caching past K, V tensors avoids recomputing them at every step (reducing total generation compute of T tokens from $\mathcal{O}(T^3 D)$ to $\mathcal{O}(T^2 D)$) and converts Attention + Linear projections from GEMM into GEMV.

3.2 Exact KV-Cache Memory Calculation (`LLM08`)

💡 Intuition (Plain English First)

How much RAM does the KV-Cache eat? Just count the dimensions of what we are storing! At every single decoder layer (out of L layers), for every sequence in the batch (B), and for every token position in the sequence (S), we must save **two vectors**: one **Key (K)** vector and one **Value (V)** vector. Each K or V vector has h_{kv} heads, each head has d_{head} numbers, and each number takes Bytes_per_element bytes (e.g., 2 Bytes for FP16/BF16, or 1 Byte for INT8). Multiply those 7 numbers together and you have the exact byte count!

📐 The Math & Every Symbol Explained

$$\text{KV Cache (Bytes)} = 2 \times B \times S \times L \times h_{kv} \times d_{\text{head}} \times \text{Bytes_per_element}$$

- **2:** One tensor for Keys (K) + one tensor for Values (V).
- **B:** Batch size (number of concurrent user sequences).
- **S:** Sequence length (prompt tokens + generated tokens cached so far).
- **L:** Number of Transformer Decoder layers.
- **h_{kv}:** Number of Key/Value heads per layer ($h_{kv} = h_q$ for MHA; $h_{kv} < h_q$ for GQA; $h_{kv} = 1$ for MQA).
- **d_{head}:** Dimension of each attention head ($d_{\text{head}} = D/h_q$).
- **Bytes_per_element:** 2 for FP16/BF16, 1 for INT8/FP8, 0.5 for INT4.
- **Binary Unit Conversion Tip:** 1 KiB $= 2^{10}$ B $= 1024$ B, 1 MiB $= 2^{20}$ B $= 1,048,576$ B, and 1 GiB $= 2^{30}$ B $= 1,073,741,824$ B.

🔢 Step-by-Step Worked Toy Example (LLM08)

> 💡 **TIP**
>
> **The Powers-of-2 Mental Math Trick:** Almost every LLM hyperparameter is a power of 2! Instead of multiplying huge numbers by hand, just **add the exponents of 2** and subtract 20 for MiB (or subtract 30 for GiB)!

Problem (LLM08): Calculate the exact KV-Cache memory in MiB for an LLM with:

- $L = 32$ layers (2^5)
- Batch size $B = 2$ (2^1)
- Sequence length $S = 2048$ (2^{11})
- Query heads $h_q = 32$, **GQA Key/Value heads** $h_{kv} = 4$ (2^2)
- Head dimension $d_{\text{head}} = 128$ (2^7)
- Precision: FP16 (2 Bytes $= 2^1$)

Step-by-Step Calculation:

1. **Step 1 — Write every factor in the formula as a power of 2:**

$$\text{Bytes} = \underbrace{2}_{2^1\ (\text{K\&V})} \times \underbrace{2}_{2^1\ (\text{B})} \times \underbrace{2048}_{2^{11}\ (\text{S})} \times \underbrace{32}_{2^5\ (\text{L})} \times \underbrace{4}_{2^2\ (h_{kv})} \times \underbrace{128}_{2^7\ (d_{\text{head}})} \times \underbrace{2}_{2^1\ (\text{FP16})}$$

2. **Step 2 — Sum the exponents of 2:**

$$1 + 1 + 11 + 5 + 2 + 7 + 1 = 28 \implies \text{Total Bytes} = 2^{28}\ \text{Bytes}$$

3. **Step 3 — Divide by 2^{20} to convert Bytes to MiB:**

$$\text{KV Cache (MiB)} = \frac{2^{28}}{2^{20}} = 2^{28-20} = 2^8 = \mathbf{256}\ \text{MiB}$$

(If this model had used standard MHA with $h_{kv} = h_q = 32$ instead of GQA with $h_{kv} = 4$, the cache would be $32/4 = 8\times$ larger: $256 \times 8 = 2048$ MiB $= 2.0$ GiB!)

🎯 Exam Takeaway & Common Traps

- **Trap in LLM08 :** Exam questions always list *both* $h_q = 32$ and $h_{kv} = 4$ to tempt you into multiplying by 32. Only h_{kv} goes into the KV-Cache formula (h_q is only used if you need to find $d_{\text{head}} = D/h_q$)! Also never forget the leading factor of 2 (one for K, one for V).

3.3 Mixed-Precision Training Memory (16Ψ Rule) & Activation Checkpointing (LLM09)

💡 Intuition (Plain English First)

Why does a 7B parameter LLM need only 14 GB of VRAM for FP16 inference, but suddenly blows up an 80 GB A100 GPU when you try to run full fine-tuning with AdamW? Because during **Mixed-Precision Training with**

AdamW, for every single parameter in the model (Ψ), you don't just store the 2-byte FP16 weight—you must also store:

1. A 2-byte FP16 **gradient**.
2. A 4-byte FP32 **master copy of the weight** (if you tried to add a tiny 10^{-5} Adam update directly to a 16-bit weight, it would round to zero and stall training!).
3. A 4-byte FP32 **first moment (m_t, momentum)** for Adam.
4. A 4-byte FP32 **second moment (v_t, uncentered variance)** for Adam.

Add those up: $2 + 2 + 4 + 4 + 4 = \mathbf{16}$ Bytes per parameter (16Ψ)! And that's *before* storing any intermediate layer activations!

📐 The Math & Every Symbol Explained

Component Stored in GPU VRAM	Precision Format	Bytes per Parameter (Ψ)	Share of Static Training VRAM
1. Forward/Backward Weights (W)	FP16 or BF16	2Ψ Bytes	12.5%
2. Weight Gradients (∇W)	FP16 or BF16	2Ψ Bytes	12.5%
3. Master Weights Copy (W_{master})	FP32	4Ψ Bytes	25.0%
4. Adam First Moment (m_t)	FP32	4Ψ Bytes	25.0%
5. Adam Second Moment (v_t)	FP32	4Ψ Bytes	25.0%
Total Static Memory (16Ψ Rule)	**Mixed (FP16/BF16 + FP32)**	**16Ψ** Bytes	100% **(75% is Adam Optimizer!)**

- **Activation Checkpointing (Gradient Checkpointing):**
  - During a standard forward pass through L layers, PyTorch saves all intermediate activations inside every layer so the backward pass can compute derivatives—costing $\mathcal{O}(L)$ activation memory.
  - With **Activation Checkpointing**, we throw away the intermediate activations inside each layer during the forward pass and only save the input at the boundary of each Transformer block. During the backward pass, we **re-run a mini forward pass** through one block at a time to regenerate its activations on the fly.
  - **Memory vs Compute Trade-off:** Slashes activation memory from $\mathcal{O}(L)$ to $\mathcal{O}(\sqrt{L})$ (or $\mathcal{O}(1)$ buffer per layer) for only $\approx +33\%$ **extra training FLOPs** (since 1 normal step $= 1$ Forward $+ 2$ Backward $= 3$ units of work, and recomputing the forward pass adds $+1$ unit $\rightarrow 4/3 = 1.33\times$).

🔢 Step-by-Step Worked Toy Example (`LLM09`)

Suppose we want to perform full Mixed-Precision (BF16 + FP32 AdamW) fine-tuning of a $\Psi = 7$ Billion parameter LLM:

1. **Step 1 — BF16 Weights + BF16 Gradients:**

$$(2 + 2) \times 7\text{B} = 4 \times 7\text{B} = \mathbf{28}\ \text{GB}$$

2. **Step 2 — FP32 AdamW Optimizer States (Master Weights + m_t + v_t):**

$$(4 + 4 + 4) \times 7\text{B} = 12 \times 7\text{B} = \mathbf{84}\ \text{GB}$$

3. **Step 3 — Total Static Memory (excluding activations):**

$$28\ \text{GB} + 84\ \text{GB} = 16 \times 7\text{B} = \mathbf{112}\ \text{GB}$$

(This immediately explains why full fine-tuning a 7B model overflows a single 80 GB GPU, and why LoRA—which freezes W and eliminates the 84 GB of Adam states on W—is revolutionary!)

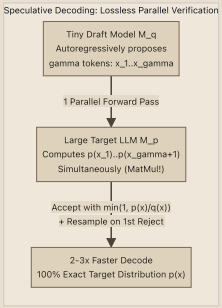
🎯 Exam Takeaway & Common Traps

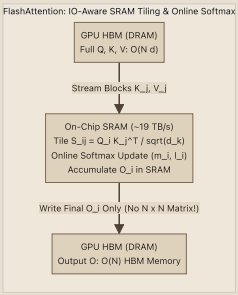
- **The 16Ψ Breakdown (`LLM09`):** Memorize $2 + 2 + (4 + 4 + 4) = 16$ Bytes/param. Notice that 12Ψ **(75%) comes purely from the FP32 AdamW optimizer states**!

3.4 FlashAttention (IO-Aware Tiling) & PagedAttention (vLLM) (`LLM10`)

💡 Intuition (Plain English First)

1. **FlashAttention:** Why is standard attention slow at $N = 8192$ tokens? Because standard PyTorch computes $S = QK^T$ (8192×8192 matrix), **writes all** 67 million **numbers out to slow GPU HBM (DRAM)**, reads them all back to compute $P = \text{softmax}(S)$, writes P back to slow HBM, and reads P back again to multiply by V! **FlashAttention** realizes that we never actually need the giant $N \times N$ matrix in slow HBM! Instead, it loads small tiles of Q, K, V into tiny, ultra-fast **on-chip SRAM** (which is $\sim$ 10–20$\times$ faster than HBM), computes the dot product, updates a running Softmax denominator using the **Online Softmax trick**, and multiplies by V right inside SRAM—writing only the final $N \times d_k$ output back to HBM!
   - *Crucial Exam Fact:* FlashAttention is **NOT an approximation**—it produces the **100% exact same mathematical output** as standard attention, while shrinking HBM memory from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$!
2. **PagedAttention (vLLM):** When serving 100 users at once, you don't know ahead of time whether User A will generate 20 tokens or 2048 tokens. Old serving engines pre-allocated a giant contiguous slab of 2048 KV-Cache slots per user, wasting 60–80% of GPU memory on empty slots (**internal fragmentation**). **PagedAttention** borrows **Virtual Memory Paging** from Operating Systems: it chops the KV-Cache into small fixed-size pages (e.g., 16 tokens per block) that can sit anywhere in physical VRAM, mapped by a lightweight Page Table!





📐 The Math & Every Symbol Explained

- **Why Online Softmax Works in FlashAttention:** Normally, $\text{softmax}(x_i) = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}$ requires knowing the row maximum $m = \max_j x_j$ across all N columns before you can divide. In **Online Softmax**, as we stream tile j across SRAM, we update the running maximum $m_{\text{new}} = \max(m_{\text{old}}, \tilde{m}_j)$ and rescale the previous running sum ℓ_{old} and output O_{old} on the fly by multiplying by the exact correction factor $e^{m_{\text{old}} - m_{\text{new}}}$!

▼ 📌 Optional Deep-Dive (Skip for Timed MCQ): FlashAttention Online Softmax Recurrence Equations

$$m_{\text{new}} = \max(m_{\text{old}}, \tilde{m}), \qquad \ell_{\text{new}} = e^{m_{\text{old}} - m_{\text{new}}} \ell_{\text{old}} + e^{\tilde{m} - m_{\text{new}}} \tilde{\ell}, \qquad O_{\text{new}} = \frac{e^{m_{\text{old}} - m_{\text{new}}} \ell_{\text{old}} O_{\text{old}} + e^{\tilde{m} - m_{\text{new}}} \tilde{P} V_j}{\ell_{\text{new}}}$$

Method	Compute Complexity (FLOPs)	HBM Memory Footprint	Is Output Mathematically Exact?
Standard Attention	$\mathcal{O}(N^2 d)$	$\mathcal{O}(N^2)$ (materializes $N \times N$ matrix S, P in HBM)	Yes
FlashAttention	$\mathcal{O}(N^2 d)$ (slightly more FLOPs due to rescaling)	$\mathcal{O}(N)$ (never writes $N \times N$ matrix to HBM!)	**Yes (100% Exact!)**

🔢 Step-by-Step Worked Toy Example

Suppose we have a row of 4 attention scores split into two tiles: Tile 1 is $[1.0,\ 2.0]$ and Tile 2 is $[4.0,\ 3.0]$:

1. **Step 1 — Process Tile 1** $[1.0,\ 2.0]$ **in SRAM:**
   - Local max $m_1 = 2.0$.
   - Local sum $\ell_1 = e^{1.0-2.0} + e^{2.0-2.0} = e^{-1} + 1 \approx 1.3679$.
2. **Step 2 — Stream Tile 2** $[4.0,\ 3.0]$ **into SRAM and update:**

- New global max $m_2 = \max(m_1, 4.0) = 4.0$.
- Rescale old sum ℓ_1 by $e^{m_1 - m_2} = e^{2.0-4.0} = e^{-2}$ and add Tile 2's sum:

  $$\ell_2 = e^{-2}(e^{-1} + 1) + (e^{4.0-4.0} + e^{3.0-4.0}) = e^{-3} + e^{-2} + 1 + e^{-1} \approx \mathbf{1.5530}$$

  We got the exact 4-element Softmax denominator $\sum_{j=1}^{4} e^{x_j - 4.0}$ in a single streaming pass without ever holding all 4 logits in memory at once!

🎯 Exam Takeaway & Common Traps

- **FlashAttention Trap (`LLM10`):** FlashAttention does **NOT** reduce theoretical FLOPs from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$! Its FLOPs are still $\mathcal{O}(N^2 d)$, and its output is **100% exact**—it runs 2–4× faster because it eliminates $\mathcal{O}(N^2)$ **HBM memory reads/writes** via SRAM tiling.

3.5 Decoding & Sampling Strategies (`LLM11`)

💡 Intuition (Plain English First)

At each decode step, the LLM outputs a raw score (logit z_i) for every word in its vocabulary V. How do we pick the next word?

- **Temperature T:** Controls how "adventurous" or "focused" the probability distribution is before sampling:
  - Dividing logits by a tiny temperature ($T \to 0^+$) magnifies differences between logits, turning Softmax into a winner-take-all spike on the single highest-scoring word (**Greedy Decoding**).
  - Dividing logits by a huge temperature ($T \to \infty$) squashes all logits toward 0, making every word in the dictionary equally likely (**Uniform Random Babble**).
- **Top-k Sampling:** Only keep the k most likely words (e.g., top 50) and set all other words to probability 0.
- **Top-p (Nucleus) Sampling:** Instead of a fixed count k, sort the words from most likely to least likely and add them to a bucket one by one **until their combined probability reaches or exceeds** p (e.g., $p = 0.90$), then stop and renormalize that bucket to sum to 1.0!

📐 The Math & Every Symbol Explained

1. **Temperature-Scaled Softmax:**

   $$p_i = \frac{\exp(z_i / T)}{\sum_{j=1}^{|V|} \exp(z_j / T)}$$

2. **Top-p (Nucleus) Set Definition:** Sort vocabulary tokens in descending order $p_{(1)} \geq p_{(2)} \geq \cdots \geq p_{(|V|)}$, and choose the **smallest integer** m such that:

   $$\sum_{i=1}^{m} p_{(i)} \geq p$$

   If combined with Top-k, we take at most $\min(m, k)$ tokens, zero out all remaining tokens, and **renormalize** the surviving tokens:

$$\hat{p}_{(i)} = \frac{p_{(i)}}{\sum_{j=1}^{\min(m,k)} p_{(j)}}$$

🔢 Step-by-Step Worked Toy Example (LLM11)

Problem (LLM11): At $T = 1.0$, a 4-token vocabulary has sorted probabilities $P = [0.50,\ 0.30,\ 0.15,\ 0.05]$. What is the final renormalized probability distribution after applying **both Top-k ($k = 2$) and Top-p ($p = 0.75$)**?

1. **Step 1 — Check cumulative probabilities for Top-p ($p = 0.75$):**
   - Include Token 1 (0.50): cumulative sum $= 0.50 < 0.75$ (not enough yet, keep going!).
   - Include Token 2 (0.30): cumulative sum $= 0.50 + 0.30 = \mathbf{0.80} \geq \mathbf{0.75}$ (stop! Nucleus size $m = 2$).
2. **Step 2 — Apply Top-k ($k = 2$) limit:**
   - Since $\min(m, k) = \min(2, 2) = 2$, Tokens 1 and 2 survive, and Tokens 3 and 4 are zeroed out.
3. **Step 3 — Renormalize the surviving probabilities by dividing by their sum (0.80):**

$$\hat{p}_1 = \frac{0.50}{0.80} = \mathbf{0.625}, \qquad \hat{p}_2 = \frac{0.30}{0.80} = \mathbf{0.375}, \qquad \hat{p}_3 = 0, \qquad \hat{p}_4 = 0$$

🎯 Exam Takeaway & Common Traps

- **Top-p Boundary Rule (LLM11):** Top-p keeps the **smallest prefix whose sum is $\geq p$** (so you *include* the token that pushes the cumulative sum across the p threshold—never stop strictly below p!).

---

4. Fine-Tuning (PEFT / LoRA) & Alignment (RLHF vs DPO vs GRPO)

↑ Mục Lục

4.1 LoRA & QLoRA Math & Exam Traps (LLM12 , LLM13)

💡 Intuition (Plain English First)

We just saw that full fine-tuning of a 7B model requires 112 GB of VRAM because storing FP32 Adam optimizer states (12Ψ) for billions of weights is massive. **LoRA (Low-Rank Adaptation)** asks a simple question: *When we fine-tune a giant* 4096×4096 *matrix* W_0 *on a new task, do we really change all* 4096 *directions independently, or does the weight update* ΔW *actually live in a tiny low-dimensional subspace (say, rank* $r = 8$ *or* 16*)?* LoRA **freezes W_0 completely** (so W_0 needs zero gradients and zero Adam optimizer states!) and represents the update ΔW as the product of two skinny matrices: a "down-projection" A (from width d_{in} down to tiny bottleneck r) and an "up-projection" B (from tiny bottleneck r back up to d_{out}).

- **Even better:** At inference time, since $W_0 x + \frac{\alpha}{r} BAx = \left(W_0 + \frac{\alpha}{r} BA\right) x$, we simply **add $\frac{\alpha}{r} BA$ directly into W_0 once before deployment**! That means LoRA adds **ZERO extra latency and ZERO extra FLOPs at inference time**!

📐 The Math & Every Symbol Explained

For a frozen pretrained weight matrix $W_0 \in \mathbb{R}^{n \times m}$ (where $n = d_{\text{out}}$ rows and $m = d_{\text{in}}$ columns) and an input vector $x \in \mathbb{R}^m$:

$$h = W_0 x + \frac{\alpha}{r} \Delta W x = W_0 x + \frac{\alpha}{r} B A x$$

- $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$**:** Frozen pretrained weight matrix (d_{out} rows, d_{in} columns; $d_{\text{out}} \cdot d_{\text{in}}$ frozen parameters).
- $A \in \mathbb{R}^{r \times d_{\text{in}}}$**:** Trainable low-rank down-projection matrix (r rows, d_{in} columns), initialized with random Gaussian noise $A \sim \mathcal{N}(0, \sigma^2)$.
- $B \in \mathbb{R}^{d_{\text{out}} \times r}$**:** Trainable low-rank up-projection matrix (d_{out} rows, r columns), initialized to **exact zeros** ($B = 0$**)**.
- $r \ll \min(d_{\text{out}}, d_{\text{in}})$**:** LoRA rank (typically $r \in \{8, 16, 32, 64\}$).
- α**:** Constant LoRA scaling hyperparameter (typically $\alpha = r$ or $\alpha = 2r$). Scaling by $\frac{\alpha}{r}$ ensures that if you change rank r, the magnitude of $\frac{\alpha}{r} BAx$ stays roughly constant without needing to re-tune learning rates!
- **Trainable Parameter Count of One LoRA Adapter:**

$$\text{Params}_{\text{LoRA}} = \text{Params}(A) + \text{Params}(B) = r \cdot d_{\text{in}} + d_{\text{out}} \cdot r = \mathbf{r} \cdot (\mathbf{d}_{\text{out}} + \mathbf{d}_{\text{in}})$$

> ⚡ **IMPORTANT**
>
> **Why MUST A be random Gaussian and $B = 0$ at initialization (`LLM12`)?**
>
> 1. **Why $B = 0$?** At training step 0, $\Delta W = BA = 0 \cdot A = \mathbf{0}$, so $h = W_0 x$. The model starts fine-tuning with the **100% exact behavior of the pretrained network**, without injecting random untrained noise!
> 2. **Why can't BOTH $A = 0$ and $B = 0$?** By the chain rule, $\frac{\partial \mathcal{L}}{\partial A} = B^T \frac{\partial \mathcal{L}}{\partial h} x^T$ and $\frac{\partial \mathcal{L}}{\partial B} = \frac{\partial \mathcal{L}}{\partial h} (Ax)^T$. If both $A = 0$ and $B = 0$, both gradients are identically 0 at step 0, trapping the network at a dead saddle point forever! With $A \neq 0$ and $B = 0$, $\frac{\partial \mathcal{L}}{\partial B} \neq 0$ on step 1, updating B to become non-zero on step 2!

- **QLoRA (4-Bit Quantized LoRA — `LLM13`):** Compresses the frozen base model W_0 to **4-bit NormalFloat (`NF4`)** while keeping the tiny LoRA adapters A, B in **BF16**:
  1. **NF4 (4-Bit NormalFloat):** Because pretrained neural network weights follow a zero-mean normal (bell-curve) distribution, NF4 places its 16 quantization bins at the **exact equal-area quantiles of** $\mathcal{N}(0, 1)$ so every bin holds an equal $1/16$th of the weights (information-theoretically optimal for normal distributions).
  2. **Double Quantization (DQ):** Quantizes the 32-bit block-wise quantization scale factors themselves down to 8-bit FP8!
  3. **Paged Optimizers:** Uses NVIDIA unified memory to automatically page AdamW optimizer states between GPU VRAM and CPU RAM during long-sequence memory spikes.

🔢 Step-by-Step Worked Toy Example (`LLM12`, `LLM13`)

1. **Problem A (`LLM12` — LoRA Parameter Count & Compression Ratio):** Suppose we attach a LoRA adapter of rank $r = 16$ to a GQA Key projection matrix $W_K \in \mathbb{R}^{1024 \times 4096}$ ($n = 1024$ output rows, $m = 4096$ input columns):
   - **Original frozen parameters in W_K:** $1024 \times 4096 = 4,194,304$.
   - **Trainable parameters in A (16×4096) and B (1024×16):**

$$\text{Params}_{\text{LoRA}} = 16 \times (1024 + 4096) = 16 \times 5120 = \mathbf{81,920} \text{ parameters}$$

- **Percentage of original matrix:** $\frac{81{,}920}{4{,}194{,}304} = \frac{16\times5120}{1024\times4096} = \frac{80}{4096} \approx \mathbf{1.95\%}$ (a $51.2\times$ reduction in trainable parameters!).

2. **Problem B (`LLM13` — QLoRA Double Quantization Exact Bit-Overhead Math):** Suppose QLoRA quantizes weights in blocks of 64 weights, storing one FP32 (32-bit) scale factor S per block:
   - **Without Double Quantization:** Each block of 64 weights pays 32 extra bits for its scale factor:

     $$\text{Scale Overhead} = \frac{32 \text{ bits}}{64 \text{ weights}} = \mathbf{0.50} \text{ bits per parameter}$$

     (So a "4-bit" model actually takes 4.50 bits/param*!)*
   - **With Double Quantization:** We quantize each block's scale factor to **8-bit FP8**, and group 256 blocks together under one second-level FP32 (32-bit) scale:

     $$\text{DQ Scale Overhead} = \frac{8 \text{ bits}}{64 \text{ weights}} + \frac{32 \text{ bits}}{64 \times 256 \text{ weights}} = 0.125 + 0.00195 \approx \mathbf{0.127} \text{ bits per parameter}$$

     (Saves $0.50 - 0.127 = \mathbf{0.373}$ bits/param*, or* ~ 3 GB *of VRAM on a 65B model!)*

🎯 Exam Takeaway & Common Traps

- **LoRA Formula (`LLM12`):** Always $r(d_{\text{out}} + d_{\text{in}})$, initialized with $A \sim \mathcal{N}(0, \sigma^2)$ and $B = 0$, merged at inference as $W_{\text{merged}} = W_0 + \frac{\alpha}{r}BA$ for **zero inference latency**.
- **QLoRA Overhead (`LLM13`):** Blocksize 64 with FP32 scales adds $32/64 = 0.5$ bits/param; Double Quantization slashes that to $8/64 + 32/(64 \times 256) \approx \mathbf{0.127}$ bits/param.

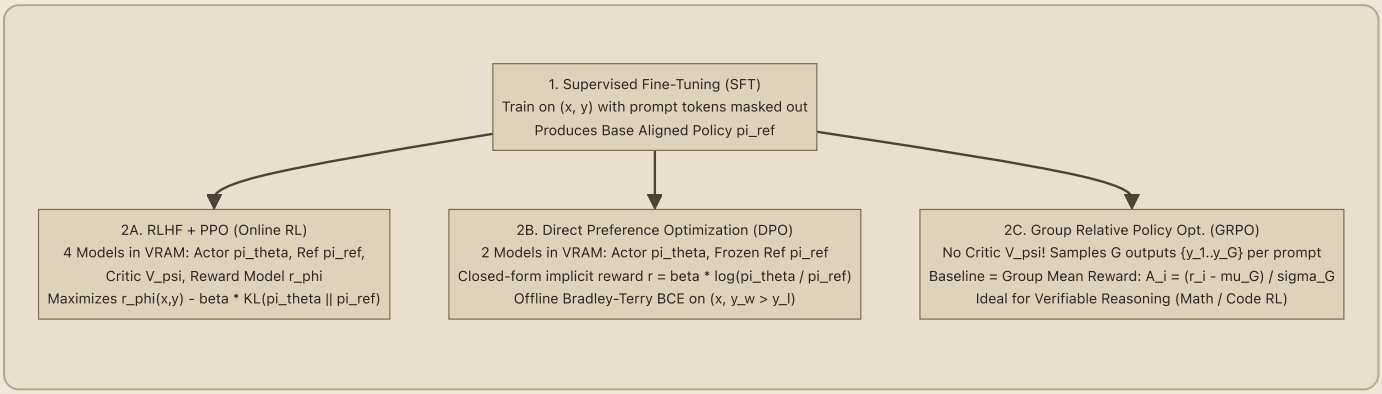
4.2 Post-Training Alignment: SFT → RLHF (PPO) vs DPO vs GRPO (`LLM14`)

💡 Intuition (Plain English First)

A pretrained base LLM only knows how to autocomplete internet text: if you ask `"How do I bake a cake?"`, it might autocomplete with `"Question 2: How do I bake a pie?"`. To turn it into a helpful, safe assistant, we run **Post-Training Alignment**:

1. **Stage 1 — Supervised Fine-Tuning (SFT):** We show the model thousands of high-quality `(User Prompt, Ideal Assistant Response)` demonstrations. *Crucial detail:* We only compute Cross-Entropy loss on the **Assistant Response tokens** (masking out the User Prompt tokens so the model isn't trained to parrot user questions!).
2. **Stage 2A — RLHF with PPO (The Classic 4-Model Pipeline):** Humans are much better at picking the better of two answers ($y_w \succ y_l$) than writing a gold answer from scratch. So we first train a **Reward Model (r_ϕ)** to score answers like a human judge. Then we use **PPO (Reinforcement Learning)** to train our LLM Policy (π_θ) to maximize the Reward Model's score.
   - *Why do we subtract a **KL-Divergence penalty** against the frozen SFT model (π_{ref})?* Without it, the LLM discovers **Reward Hacking**: it outputs bizarre gibberish like `"helpful helpful safe A+ 100%"` that tricks the Reward Model into giving a high score!
   - *Downside of PPO:* You must hold **4 full LLMs in GPU memory at once**: Actor (π_θ), Reference (π_{ref}), Critic/Value network (V_ψ), and Reward Model (r_ϕ).

3. **Stage 2B — DPO (Direct Preference Optimization — 2 Models, No RL!):** Rafailov et al. (2023) proved a stunning mathematical shortcut: you can solve the KL-constrained reward equation for the reward $r(x, y)$ directly in terms of the LLM's own probabilities $\log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$! That means **the LLM itself IS the reward model**! DPO throws away the Reward Model and Critic completely, training directly on (x, y_w, y_l) pairs with a simple binary cross-entropy loss using only **2 models in VRAM** (π_θ and frozen π_{ref}).
4. **Stage 2C — GRPO (Group Relative Policy Optimization — DeepSeek-R1):** What if you *need* online RL (for verifiable math or code problems where a compiler/checker grades the answer), but PPO's Critic network V_ψ is too big for VRAM? **GRPO** deletes the Critic network V_ψ! Instead, for each prompt x, it samples a **group of G answers** $\{y_1, \ldots, y_G\}$ and uses the **group's average reward** as the baseline!



📐 The Math & Every Symbol Explained

1. **Reward Model Bradley-Terry Loss (for preference pair $y_w \succ y_l$ given prompt x):**

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x,y_w,y_l)}\Big[\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)\Big]$$

   - y_w **("winner") and** y_l **("loser"):** The human-preferred and human-dispreferred responses to prompt x.
   - $\sigma(z) = \frac{1}{1+e^{-z}}$**:** Logistic sigmoid function mapping the score difference $r_\phi(x, y_w) - r_\phi(x, y_l)$ into a probability in $(0, 1)$.

2. **RLHF (PPO) KL-Regularized Objective:**

$$\max_{\pi_\theta} \ \mathbb{E}_{x\sim\mathcal{D},\ y\sim\pi_\theta(\cdot|x)}\Big[r_\phi(x, y) - \beta\, D_{\text{KL}}\big(\pi_\theta(y \mid x) \,\|\, \pi_{\text{ref}}(y \mid x)\big)\Big]$$

   - $\beta > 0$**:** KL penalty coefficient controlling how far the actor π_θ is allowed to drift from the frozen SFT reference policy π_{ref}.

3. **DPO (Direct Preference Optimization) Loss (`LLM14`):** Substituting the closed-form implicit reward $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ into the Bradley-Terry loss cancels the partition function $Z(x)$ and yields:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

4. **GRPO (Group Relative Policy Optimization) Advantage Formula:** For a group of G sampled outputs $\{y_1, \ldots, y_G\}$ with rewards $\{r_1, \ldots, r_G\}$:

$$\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G + \epsilon}, \qquad \text{where } \mu_G = \frac{1}{G}\sum_{j=1}^{G} r_j, \quad \sigma_G = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(r_j - \mu_G)^2}$$

🔢 Step-by-Step Worked Toy Example

Let's trace **GRPO Advantage Calculation** on a math reasoning prompt x where we sample $G = 4$ responses from our LLM and check them with a Python math verifier ($r = 1$ if correct, $r = 0$ if wrong). Suppose only Response 1 is correct, so the rewards are $R = [1.0,\ 0.0,\ 0.0,\ 0.0]$:

1. **Step 1 — Compute the Group Mean Reward μ_G:**

$$\mu_G = \frac{1.0 + 0.0 + 0.0 + 0.0}{4} = 0.25$$

2. **Step 2 — Compute the Group Standard Deviation σ_G ($\epsilon = 0$):**

$$\sigma_G = \sqrt{\frac{(1.0 - 0.25)^2 + 3 \times (0.0 - 0.25)^2}{4}} = \sqrt{\frac{0.5625 + 0.1875}{4}} = \sqrt{0.1875} \approx 0.433$$

3. **Step 3 — Compute the Group-Normalized Advantage $\hat{A}_i$ for each response:**
   - Correct response ($r_1 = 1.0$): $\hat{A}_1 = \frac{1.0 - 0.25}{0.433} = +\mathbf{1.732}$ (reinforced!).
   - Wrong responses ($r_{2,3,4} = 0.0$): $\hat{A}_{2,3,4} = \frac{0.0 - 0.25}{0.433} = -\mathbf{0.577}$ (penalized!). *(Notice how GRPO computed exact positive and negative RL advantages without needing a 7B Critic neural network V_ψ!)*

🎯 Exam Takeaway & Common Traps

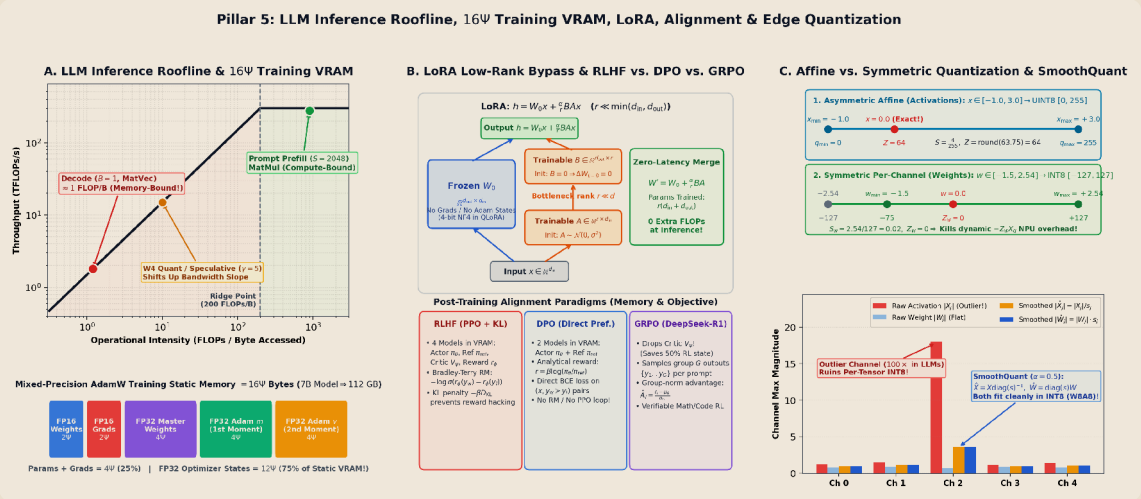
- **Alignment Comparison (`LLM14`):**
  - **RLHF (PPO):** Online RL, requires **4 models** (Actor, Ref, Critic, Reward); KL penalty prevents **reward hacking / mode collapse**.
  - **DPO:** Offline preference learning, requires **2 models** (Actor, Ref); derives implicit reward $r(x, y) = \beta \log \frac{\pi_\theta}{\pi_{\text{ref}}}$ via Bradley-Terry substitution.
  - **GRPO:** Online RL without a Critic network; normalizes rewards across a group of G sampled outputs per prompt.

---

5. Edge AI, Quantization, Distillation & Fast Inference (Qualcomm Focus)

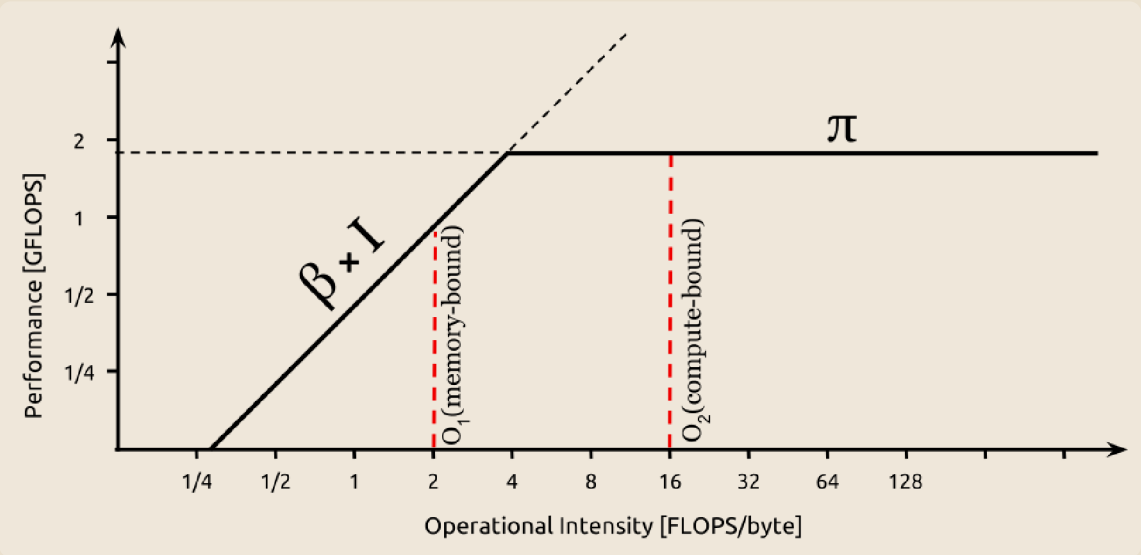
↑ Mục Lục

FIGURE SLIDE 1 OF 2



LLM Inference Roofline, 16-Psi Training VRAM, LoRA, Alignment & Edge Quantization

FIGURE SLIDE 2 OF 2



Canonical Roofline Model: Memory-Bandwidth-Bound vs Compute-Bound Regimes

5.1 Floating-Point Formats: FP32 vs FP16 vs BF16 (`LLM15`)

💡 Intuition (Plain English First)

Every floating-point number in a computer is stored in scientific notation using three parts:

$$\text{Value} = (-1)^{\text{Sign}} \times 2^{(\text{Exponent} - \text{Bias})} \times (1.\text{Mantissa})_2$$

- **The Exponent bits** control the **Dynamic Range** (how huge or how tiny a number can be before overflowing to ∞ or underflowing to 0.0).

- **The Mantissa (Fraction) bits** control the **Precision** (how many decimal digits of accuracy you get between two powers of 2).

When engineers first tried training deep networks in **IEEE FP16** (1 sign, **5 exponent**, 10 mantissa), training crashed constantly! Why? Because a **5-bit exponent** can only represent numbers up to $65,504$ and down to $\sim 6 \times 10^{-8}$, so tiny gradients underflowed to 0.0 unless you added complicated **Dynamic Loss Scaling**. Google Brain invented **BF16 (Brain Float 16)** by taking a 32-bit FP32 number (1 sign, **8 exponent**, 23 mantissa) and simply chopping off the last 16 mantissa bits (leaving 1 sign, **8 exponent**, 7 mantissa)! Because BF16 keeps all **8 exponent bits** of FP32, it has the **exact same dynamic range as FP32** ($\sim 10^{\pm 38}$) and never needs loss scaling!

📐 The Math & Every Symbol Explained

Format	Total Bits	Sign Bits	Exponent Bits (Dynamic Range)	Mantissa Bits (Precision)	Key Hardware & Deep Learning Behavior
FP32	32	1	**8 bits** ($\sim 10^{\pm 38}$)	23 bits	Standard master-weight & accumulation precision
FP16	16	1	**5 bits** (max $= 65,504$)	10 bits	Higher precision than BF16, but **narrow range** $\rightarrow$ gradients underflow; requires **Dynamic Loss Scaling**
BF16	16	1	**8 bits** ($\sim 10^{\pm 38}$)	7 bits	**Identical dynamic range to FP32**; drop-in replacement for LLM training without loss scaling!
INT8	8	1 (signed)	0 (Uniform Integer $[-128, 127]$ or $[0, 255]$)	0	$4\times$ smaller than FP32; native fast integer MACs on Qualcomm Hexagon NPU
INT4	4	1 (signed)	0 (Uniform Integer $[-8, 7]$ or $[0, 15]$)	0	$8\times$ smaller than FP32; standard for on-device LLM weights (`W4A16` / `W4A8`)

🔢 Step-by-Step Worked Toy Example

Suppose a backward pass produces a gradient value of $g = 10^{-10}$, and a giant activation spike reaches $a = 100,000$:

1. **In FP16 (5-bit exponent, range $\approx [6 \times 10^{-8},\ 65,504]$):**
   - The gradient $g = 10^{-10} < 6 \times 10^{-8}$ **underflows to** 0.0!
   - The activation $a = 100,000 > 65,504$ **overflows to** `Inf` (`NaN`)!
2. **In BF16 (8-bit exponent, range $\approx [1.2 \times 10^{-38},\ 3.4 \times 10^{38}]$):**
   - Both $g = 10^{-10}$ and $a = 100,000$ fit comfortably inside $[10^{-38}, 10^{38}]$ with **zero underflow and zero overflow**!

🎯 Exam Takeaway & Common Traps

- **Memorize the Bit Splits (`LLM15`):**
  - **FP32:** 1 sign + 8 exp + 23 mantissa.
  - **FP16:** 1 sign + 5 exp + 10 mantissa (needs loss scaling!).
  - **BF16:** 1 sign + 8 exp + 7 mantissa (same range as FP32!).

5.2 Exact Quantization Math: Asymmetric (Affine) vs Symmetric (`LLM16`, `LLM17`)

💡 Intuition (Plain English First)

How do we squeeze continuous floating-point numbers $x \in [x_{\min}, x_{\max}]$ into 256 discrete 8-bit integer buckets $x_q \in [q_{\min}, q_{\max}]$? Think of it like converting temperatures between Celsius and Fahrenheit using a **step size (Scale S)** and an **offset (Zero-Point Z)**:

1. **Asymmetric (Affine) Quantization (`LLM16`):** Maps the exact minimum float $x_{\min}$ to the bottom integer $q_{\min}$ (e.g., 0) and the exact maximum float $x_{\max}$ to the top integer $q_{\max}$ (e.g., 255). This uses 100% of all 256 buckets, even if your data is lopsided (like ReLU activations $x \in [-1.0, +3.0]$).
   - *Why is Zero-Point Z rounded to an exact integer?* Because 0.0 appears everywhere in neural networks (zero-padding around CNN images, ReLU zeros, causal masks). If 0.0 maps to an exact integer Z, our hardware can represent 0.0 with **zero quantization error**!
2. **Symmetric Quantization (`LLM17`):** Forces the Zero-Point to be $Z = 0$ (so float 0.0 always maps to integer 0) and stretches the range symmetrically to $\pm \max(|x_{\min}|, |x_{\max}|)$ across $[-127, +127]$.
   - *Why do Qualcomm Hexagon NPUs love Symmetric Quantization for Weights ($Z_w = 0$), even if it wastes a few buckets?* Because when you multiply quantized inputs $(X_q - Z_x)$ by quantized weights $(W_q - Z_w)$, expanding the product creates a cross-term $-Z_w X_q$. Since X_q changes dynamically on every single user query at runtime, if $Z_w \neq 0$, the NPU has to waste extra clock cycles summing up the rows of X_q at runtime! Setting $Z_w = 0$ **kills that runtime overhead completely**!

📐 The Math & Every Symbol Explained

1. **Asymmetric (Affine) Quantization (`LLM16`):**

$$S = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \qquad Z = \text{round}\left(q_{\min} - \frac{x_{\min}}{S}\right)$$

$$\text{Quantize: } x_q = \text{clamp}\left(\text{round}\left(\frac{x}{S}\right) + Z,\ q_{\min},\ q_{\max}\right), \qquad \text{Dequantize: } \hat{x} = S \cdot (x_q - Z)$$

   - $[x_{\min}, x_{\max}]$**:** Observed floating-point range of the tensor.
   - $[q_{\min}, q_{\max}]$**:** Target integer grid ($[0, 255]$ for `UINT8` where $q_{\max} - q_{\min} = 255$; $[-128, 127]$ for `INT8` where $q_{\max} - q_{\min} = 255$).
   - S **(Scale):** Floating-point step size per integer tick.
   - Z **(Zero-Point):** Exact integer value that corresponds to real floating-point 0.0.
2. **Symmetric Quantization ($Z = 0$, `LLM17`):** Using the symmetric signed grid $[-q_{\max}, +q_{\max}]$ (for `INT8`, $[-127, +127]$ so $q_{\max} = 127$; for `INT4`, $[-7, +7]$ so $q_{\max} = 7$):

$$S = \frac{\max(|x_{\min}|,\ |x_{\max}|)}{q_{\max}}, \qquad Z = 0$$

$$\text{Quantize: } x_q = \text{clamp}\left(\text{round}\left(\frac{x}{S}\right),\ -q_{\max},\ +q_{\max}\right), \qquad \text{Dequantize: } \hat{x} = S \cdot x_q$$

3. **The Hardware NPU Matrix Multiplication Expansion (Why $Z_w = 0$ Matters):**

$$\hat{Y} = S_x S_w (X_q - Z_x)(W_q - Z_w) = S_x S_w \Big[\underbrace{X_q W_q}_{\text{INT8 MACs}} - \underbrace{(Z_x W_q - Z_x Z_w)}_{\text{Folded into Bias Offline}} - \underbrace{Z_w X_q}_{\text{Runtime Penalty } (=0 \text{ if } Z_w=0)} \Big]$$

4. **Quantization Granularity Golden Rule:**
   - **Per-Tensor:** 1 scale S for the whole tensor (standard for **Activations**, or **Per-Token** in LLMs).
   - **Per-Channel (Per-Output-Channel):** 1 scale S_i per output row of W (standard for **CNN / Linear INT8 Weights** because different output channels have wildly different weight ranges).
   - **Per-Group (e.g., group size $g = 128$):** 1 scale per 128 contiguous weights along a row (standard for **INT4 LLM Weights** in GPTQ / AWQ).

🔢 Step-by-Step Worked Toy Example (`LLM16` , `LLM17`)

1. **Problem A (`LLM16` — Asymmetric `UINT8` Quantization by Hand):** Quantize an activation tensor with range $[x_{\min}, x_{\max}] = [-1.0,\ +3.0]$ to unsigned 8-bit integer `UINT8` ($[q_{\min}, q_{\max}] = [0, 255]$):
   - **Step 1 — Compute Scale S:**

     $$S = \frac{3.0 - (-1.0)}{255 - 0} = \frac{4.0}{255} \approx \mathbf{0.015686}$$

   - **Step 2 — Compute Zero-Point Z:**

     $$Z = \text{round}\left(0 - \frac{-1.0}{4.0/255}\right) = \text{round}\left(\frac{255}{4}\right) = \text{round}(63.75) = \mathbf{64}$$

   - **Step 3 — Verify that real $x = 0.0$ has zero quantization error:**

     $$x_q(0.0) = \text{round}\left(\frac{0.0}{S}\right) + 64 = 64 \implies \hat{x}(0.0) = S \cdot (64 - 64) = \mathbf{0.0} \quad \text{(Exact!)}$$

2. **Problem B (`LLM17` — Symmetric `INT8` Weight Quantization by Hand):** Quantize a weight tensor with range $[w_{\min}, w_{\max}] = [-1.50,\ +2.54]$ using Symmetric `INT8` on $[-127, +127]$ ($q_{\max} = 127$):
   - **Step 1 — Find the maximum absolute value:**

     $$\max(|-1.50|,\ |2.54|) = 2.54$$

   - **Step 2 — Compute Scale S_w and Zero-Point Z_w:**

     $$S_w = \frac{2.54}{127} = \mathbf{0.02}, \qquad Z_w = \mathbf{0}$$

   - **Step 3 — Quantize weight $w = -1.50$:**

     $$w_q = \text{round}\left(\frac{-1.50}{0.02}\right) = \text{round}(-75.0) = \mathbf{-75}$$

🎯 Exam Takeaway & Common Traps

- **Asymmetric vs Symmetric Summary (`LLM16` , `LLM17`):**

- **Activations** (like post-ReLU or post-GELU/Swish) are lopsided → use **Asymmetric Quantization** ($Z_x \neq 0$).
- **Weights** are roughly zero-centered and static → use **Per-Channel Symmetric Quantization** ($Z_w = 0$) so the dynamic cross-term $-Z_w X_q$ vanishes at runtime!

5.3 PTQ vs QAT & Modern LLM Quantization (GPTQ, AWQ, SmoothQuant) (`LLM18`)

💡 Intuition (Plain English First)

When we quantize a model, we have two paradigms:

1. **PTQ (Post-Training Quantization):** Take a finished model, pass a tiny calibration set (128–512 sentences) through it in a few minutes, and quantize without retraining.
   - **Why do naive PTQ methods fail on LLMs at 4-bit (`W4A16`) or 8-bit (`W8A8`)?** Because in LLMs larger than $\sim$ 6.7B parameters, a few specific **activation channels develop massive outliers (**$100\times$ **larger than normal channels)** across every token!
   - **How the Big Three LLM PTQ Algorithms solve this (`LLM18`):**
     - **GPTQ (`W4A16`):** Quantizes weights column-by-column using the **inverse Hessian** $H^{-1} = (2XX^T)^{-1}$ (second-order Taylor curvature of layer error $\|WX - \hat{W}X\|_2^2$) to slightly adjust the remaining unquantized columns so they cancel out the rounding error of the column just quantized!
     - **AWQ (Activation-Aware Weight Quantization, `W4A16`):** Notices that the $\sim 1\%$ of weight channels that multiply those huge **outlier activation channels** are $100\times$ more important ("salient")! Instead of keeping those 1% weights in FP16 (which ruins GPU/NPU SIMD memory alignment), AWQ **multiplies those salient weight channels by a scale** $s > 1$ **before INT4 quantization** (and divides the corresponding input activation channel by s) so their rounding error shrinks!
     - **SmoothQuant (`W8A8`):** If you want fast **INT8 Matrix Multiplication (`W8A8`)**, those $100\times$ activation outliers ruin per-tensor INT8 activation quantization (squashing normal channels to zero), while the weights are super flat and easy to quantize. **SmoothQuant** divides the outlier activation channels by a smoothing factor s_j and multiplies the corresponding weight channels by the same s_j! Mathematically, $(X\text{diag}(s)^{-1})(\text{diag}(s)W) = XW$ is unchanged, but now the activations are smooth enough for INT8!
2. **QAT (Quantization-Aware Training):** If PTQ still loses accuracy (e.g., pushing down to `W4A4` or `INT2`), we fine-tune the model with **Fake Quantization** nodes inserted in the forward pass so the weights learn to survive rounding noise.
   - **Wait—the rounding function** $\text{round}(u)$ **is a staircase whose derivative is** 0 **everywhere (and** ∞ **at the steps)! How can gradients flow backward through** $\text{round}(u)$**?** We use the **Straight-Through Estimator (STE)**: during the forward pass we run the real staircase $\text{round}(u)$, but during the backward pass we **pretend** $\text{round}(u)$ **is the identity line** $f(u) = u$ (derivative $= 1$ inside the clamp range $[q_{\min}, q_{\max}]$, and 0 outside)!

📐 The Math & Every Symbol Explained

Method	Target Format	Core Mathematical Mechanism	Key Problem Solved
GPTQ	`W4A16` (Weight-only INT4)	Uses inverse Hessian $H^{-1} = (2XX^T)^{-1}$ to update remaining weights as columns are quantized	Minimizes layer-wise output squared error $\|WX - \hat{W}X\|_F^2$
AWQ	`W4A16` (Weight-only INT4)	Scales up the 1% salient weight channels determined by **activation magnitude** $\|X_j\|$, not weight magnitude	Protects salient channels in uniform INT4 without slow mixed-precision hardware
SmoothQuant	`W8A8` (INT8 Weights & Activations)	Per-channel diagonal migration: $\hat{X} = X\text{diag}(s)^{-1}$, $\hat{W} = \text{diag}(s)W$ with $s_j = \frac{\max(\|X_j\|)^\alpha}{\max(\|W_j\|)^{1-\alpha}}$	Migrates 100× outlier difficulty from activations to weights so both fit in INT8
QAT + STE	Ultra-low bit (`W4A8`, `W4A4`, `W2`)	Forward: $\hat{x} = S(\text{clamp}(\text{round}(x/S) + Z))$. Backward **STE**: $\frac{\partial \text{round}(u)}{\partial u} \equiv 1$ on $[q_{\min}, q_{\max}]$	Bypasses the zero-everywhere derivative of round(·) during backpropagation

🔢 Step-by-Step Worked Toy Example (`LLM18`)

Suppose Channel j of activation X_j has a huge outlier peak $\max(|X_j|) = 100.0$, while the corresponding weight channel W_j is tiny with peak $\max(|W_j|) = 0.25$. Let's apply **SmoothQuant** with $\alpha = 0.5$:

1. **Step 1 — Compute the SmoothQuant per-channel migration factor s_j:**

$$s_j = \frac{\max(|X_j|)^{0.5}}{\max(|W_j|)^{1-0.5}} = \frac{\sqrt{100.0}}{\sqrt{0.25}} = \frac{10.0}{0.5} = \mathbf{20.0}$$

2. **Step 2 — Smooth the activation channel $\hat{X}_j = X_j/s_j$ and weight channel $\hat{W}_j = s_j W_j$:**
   - New activation peak: $\max(|\hat{X}_j|) = \frac{100.0}{20.0} = \mathbf{5.0}$ (shrunk by 20×!).
   - New weight peak: $\max(|\hat{W}_j|) = 20.0 \times 0.25 = \mathbf{5.0}$ (balanced to the exact same max of 5.0!).

3. **Step 3 — Check that the matrix product is 100% preserved:**

$$\hat{X}_j \hat{W}_j = \left(\frac{X_j}{20.0}\right)(20.0\, W_j) = X_j W_j$$

   Now both $\hat{X}$ and $\hat{W}$ have a gentle maximum of 5.0 and quantize cleanly into `INT8`!

🎯 Exam Takeaway & Common Traps

- **AWQ Trap (`LLM18`):** How does AWQ choose which 1% of weight channels are salient? By looking at the **activation magnitudes** $\|X_j\|$, NOT the weight magnitudes $\|W_j\|$!
- **STE Trap:** Outside the clamping range $[q_{\min}, q_{\max}]$, the STE gradient is 0 (clipped), and inside $[q_{\min}, q_{\max}]$ it is 1.

5.4 Knowledge Distillation, Pruning, Speculative Decoding & RAG (`LLM19`, `LLM20`)

💡 Intuition (Plain English First)

Let's master the final four edge-efficiency pillars tested in `LLM19`–`LLM20`:

1. **Knowledge Distillation (T^2 Rule):** A small Student model learns much more from a giant Teacher model's **soft probability distribution** (e.g., "*this image is* 70% *dog,* 29% *wolf,* 1% *car*") than from a hard 0/1 label ("100% *dog*"). To reveal those dark-knowledge probabilities on the non-winning classes, we soften both Teacher and Student logits with a high temperature $T > 1$ (e.g., $T = 4$).
   - *Why must we multiply the KL Distillation Loss by T^2?* Because dividing logits by T inside Softmax shrinks the backward gradient w.r.t. the student logits by a factor of $1/T$ from the student Softmax and another factor of $1/T$ from the logit difference, totaling $1/T^2$! Multiplying by T^2 restores the gradient magnitude so it doesn't get drowned out by the hard-label Cross-Entropy loss!
2. **Pruning (Unstructured vs Structured vs 2:4 Sparse Tensor Cores — `LLM19`):**
   - **Unstructured Pruning:** Zeroes out the smallest 50% of weights scattered randomly across a matrix. Sounds great on paper, right? **Trap:** On a GPU or NPU, dense matrix-multiply hardware still loads and multiplies every single 0.0, giving **ZERO wall-clock speedup** at 50% sparsity!
   - **Structured Pruning:** Deletes entire rows/columns (entire attention heads or FFN channels), physically shrinking the matrix from 4096×4096 to 2048×4096 for an immediate speedup on *any* hardware.
   - **NVIDIA / Hardware 2:4 Semi-Structured Sparsity:** Every group of 4 contiguous weights must contain **at least 2 zeros** (50% sparsity). Hardware Sparse Tensor Cores use a tiny 2-bit index mux to skip the two zeros entirely, delivering an **exact $2\times$ math speedup**!
3. **Speculative Decoding (Lossless 2–3$\times$ Speedup):** Remember from Section 3.1 that generating 1 token at a time with a 70B Target Model M_p is memory-bandwidth-bound because you haul 140 GB of weights from DRAM just to process 1 token. What if you haul those 140 GB of weights once and process 5 **tokens at the same time**? It takes almost the **exact same wall-clock time** as 1 token! **Speculative Decoding** uses a tiny 1B **Draft Model** M_q to quickly guess $\gamma = 5$ future tokens $[\tilde{x}_1, \ldots, \tilde{x}_5]$, and then runs the giant 70B **Target Model** M_p on all 5 tokens **in 1 single parallel forward pass** to verify them! Using a clever rejection-sampling rule $\min\left(1, \frac{p(x)}{q(x)}\right)$, the final generated text is **100% mathematically identical to sampling from the giant Target Model alone (Lossless!)**.
4. **RAG (Retrieval-Augmented Generation): Bi-Encoder vs Cross-Encoder (`LLM20`):** When searching 10 Million company documents for a user query:
   - **Stage 1 — Bi-Encoder (Dense Retriever):** Encodes the Query $E_Q(q)$ and each Document $E_D(d_i)$ **separately** into single vectors. Because all 10M document vectors are precomputed offline, finding the Top-100 closest vectors via dot-product search (HNSW / FAISS) takes < 5 milliseconds!
   - **Stage 2 — Cross-Encoder (Reranker):** Concatenates `[CLS] Query [SEP] Document [SEP]` and runs full token-by-token cross-attention across the Top-100 candidates. Much more accurate than a Bi-Encoder, but **impossible to precompute** (running it on all 10M docs would take hours per query!).

📐 The Math & Every Symbol Explained

1. **Knowledge Distillation Objective with T^2 Scaling:**

$$\mathcal{L}_{\text{KD}} = (1-\alpha)\,\mathcal{L}_{\text{CE}}\big(y,\ \sigma(z_s)\big) + \alpha\,\mathbf{T^2}\,D_{\text{KL}}\Big(\sigma(z_t/T)\ \|\ \sigma(z_s/T)\Big)$$

▼ 📌 **Optional Deep-Dive (Skip for Timed MCQ): Taylor Expansion Proof of 1/T² Gradient Shrinkage**

When T is large compared to the zero-mean logits z_i, Taylor expanding $e^{z_i/T} \approx 1 + \frac{z_i}{T}$ gives:

$$\frac{\partial D_{\text{KL}}}{\partial z_{s,i}} = \frac{1}{T}(p_{s,i}(T) - p_{t,i}(T)) \approx \frac{1}{T}\left(\frac{1 + z_{s,i}/T}{C} - \frac{1 + z_{t,i}/T}{C}\right) = \frac{1}{C \cdot \mathbf{T^2}}(z_{s,i} - z_{t,i})$$

Multiplying the KL loss by T^2 cancels the $1/T^2$ denominator!

2. **Speculative Decoding Acceptance & Residual Distribution Rule:** For draft token $\tilde{x} \sim q(x)$ proposed by Draft Model M_q and evaluated by Target Model M_p with probability $p(\tilde{x})$:

$$\text{Accept Probability} = \min\left(1,\ \frac{p(\tilde{x})}{q(\tilde{x})}\right)$$

If rejected at step i, we immediately resample a corrected token x_i' from the **residual distribution**:

$$p_{\text{residual}}(x) = \frac{\max\left(0,\ p(x) - q(x)\right)}{\sum_{v \in V} \max\left(0,\ p(v) - q(v)\right)}$$

🔢 Step-by-Step Worked Toy Example

Let's trace **Speculative Decoding Verification** on a 2-word vocabulary $V = \{\text{"cat"}, \text{"dog"}\}$ where at step 1:

- The **Target Model** M_p has true distribution $p = [0.80,\ 0.20]$.
- The **Draft Model** M_q has approximate distribution $q = [0.50,\ 0.50]$. Suppose the Draft Model samples $\tilde{x}_1$:

1. **Case A — Draft proposes `"cat"` (happens with draft probability $q(\text{"cat"}) = 0.50$):**
   - Acceptance probability: $\min\left(1, \frac{p(\text{"cat"})}{q(\text{"cat"})}\right) = \min\left(1, \frac{0.80}{0.50}\right) = \min(1, 1.6) = \mathbf{1.0}$ (100% accepted!).
2. **Case B — Draft proposes `"dog"` (happens with draft probability $q(\text{"dog"}) = 0.50$):**
   - Acceptance probability: $\min\left(1, \frac{p(\text{"dog"})}{q(\text{"dog"})}\right) = \min\left(1, \frac{0.20}{0.50}\right) = \mathbf{0.40}$ (40% accepted, 60% rejected!).
   - Overall probability of accepting `"dog"` $= q(\text{"dog"}) \times 0.40 = 0.50 \times 0.40 = \mathbf{0.20} = p(\text{"dog"})$!
   - And if `"dog"` is rejected (which happens with probability $0.50 \times 0.60 = 0.30$), we resample from $\max(0, p - q) = [\max(0, 0.8 - 0.5),\ \max(0, 0.2 - 0.5)] = [0.30,\ 0.0] \xrightarrow{\text{norm}} [1.0,\ 0.0]$ (always picks `"cat"`!).
   - **Total probability of outputting `"cat"`:** 0.50 (direct accept) + 0.30 (resampled after reject) $= \mathbf{0.80} = p(\text{"cat"})$! The final output probabilities $[0.80,\ 0.20]$ match the giant Target Model M_p **to the exact decimal place**!

🎯 Exam Takeaway & Common Traps

- **Pruning Trap (`LLM19`):** 50% unstructured magnitude pruning gives 0× **wall-clock speedup** on dense GEMM hardware; use **Structured Pruning** or **2:4 Semi-Structured Sparsity** for real hardware acceleration.
- **Speculative Decoding Fact:** Converts memory-bound Decode ($S = 1$ MatVec) into parallel verification ($S = \gamma + 1$ MatMul) and is **100% mathematically lossless**.
- **RAG Pipeline (`LLM20`):** Always **Bi-Encoder first** (fast offline-indexed MIPS over millions of docs) → **Cross-Encoder second** (accurate joint self-attention reranking over top-K docs).

6. Master Quick-Reference Checklist (`LLM01` – `LLM20`)

↑ Mục Lục

Question ID	Core Concept Tested	The 10-Second Formula / Golden Rule
`LLM01`	Scaled Dot-Product Attention Variance	If $\text{Var}(q_i) = \text{Var}(k_i) = \sigma^2$, then $\text{Var}(q \cdot k) = \sigma^4 d_k$; divide by $\sigma^2\sqrt{d_k}$ to restore unit variance ($\sqrt{d_k}$ when $\sigma^2 = 1$).
`LLM02`	Multi-Head Attention (MHA) Head Count h	Changing h while keeping $D = h \cdot d_k$ constant changes parameter count ($4D^2$) by 0, but scales (B, h, N, N) score memory linearly with h.
`LLM03`	MHA FLOPs & Crossover Sequence Length	Linear QKVO projections $= 8ND^2$ FLOPs; Quadratic Attention $= 4N^2D$ FLOPs; equal at $N = 2D$.
`LLM04`	Causal Attention Masking & Pre-LN	Add $-\infty$ (never 0) to upper triangle $j > i$ *before* Softmax so $e^{-\infty} = 0$. Pre-LN creates an unobstructed identity gradient highway.
`LLM05`	LLaMA Decoder Layer Parameter Count	Params $= \underbrace{2D^2 + 2Dh_{kv}d_k}_{\text{GQA}} + \underbrace{3DD_{\text{ff}}}_{\text{SwiGLU}} + \underbrace{2D}_{\text{2 RMSNorms}}$ (`bias=False`).
`LLM06`	MHA vs MQA vs GQA KV-Cache Ratio	GQA shares 1 K/V head across h_q/h_{kv} Query heads, slashing KV-Cache size by exactly h_q/h_{kv}.
`LLM07`	RoPE (Rotary Position Embedding)	Rotates 2D pairs of Q **and** K **only** (never V) by $m\theta_i$; orthogonal rotation preserves L_2 norm and makes $q_m^T k_n$ depend only on $m - n$.
`LLM08`	Exact KV-Cache Memory Formula	Bytes $= 2 \times B \times S \times L \times h_{kv} \times d_{\text{head}} \times$ Bytes_per_elem (use powers of 2: divide by 2^{20} for MiB, 2^{30} for GiB).
`LLM09`	Mixed-Precision AdamW Training Memory	16Ψ Bytes static VRAM: 2Ψ FP16 weights $+2\Psi$ FP16 grads $+4\Psi$ FP32 master weights $+4\Psi$ FP32 $m_t + 4\Psi$ FP32 v_t.
`LLM10`	Roofline Model & FlashAttention	Prefill is **Compute-Bound** (GEMM); Decode is **Memory-Bandwidth-Bound** (GEMV, OI ≈ 1). FlashAttention is **100% exact** and cuts HBM I/O to $\mathcal{O}(N)$ via SRAM tiling.
`LLM11`	Top-k & Top-p (Nucleus) Sampling	Sort probabilities descending, keep smallest prefix with cumulative sum $\geq p$ (capped at k tokens), and renormalize by dividing by their sum.
`LLM12`	LoRA Math, Initialization & Merging	$\Delta W = \frac{\alpha}{r}BA$ has $r(d_{\text{out}} + d_{\text{in}})$ params; init $A \sim \mathcal{N}(0, \sigma^2), B = 0$ so $\Delta W = 0$ at step 0; merge $W_0 + \frac{\alpha}{r}BA$ for 0 **inference latency**.
`LLM13`	QLoRA: NF4 & Double Quantization	NF4 is quantile-optimal for normal weights; Double Quantization cuts block-64 scale overhead from $32/64 = 0.50$ down to ≈ 0.127 bits/param.
`LLM14`	Alignment: RLHF (PPO) vs DPO vs GRPO	PPO needs **4 models** + KL penalty against reward hacking; DPO needs **2 models** (implicit reward $\beta \log \frac{\pi_\theta}{\pi_{\text{ref}}}$); GRPO drops the Critic via **group-mean baseline**.
`LLM15`	Floating-Point Bit Layouts	**FP32:** $1 + 8 + 23$; **FP16:** $1 + 5 + 10$ (narrow range, needs loss scaling); **BF16:** $1 + 8 + 7$ (**same dynamic range as FP32**).
`LLM16`	Asymmetric (Affine) Quantization	$S = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}$, $Z = \text{round}(q_{\min} - x_{\min}/S)$; integer Z guarantees **exact** 0.0 **representation** for padding/ReLU.
`LLM17`	Symmetric Quantization & NPU Hardware	$S = \frac{\max(\lvert x_{\min}\rvert, \lvert x_{\max}\rvert)}{q_{\max}}$, $Z_w = 0$; eliminates the dynamic runtime cross-term $-Z_w X_q$ in integer GEMM!
`LLM18`	GPTQ, AWQ, SmoothQuant & QAT STE	**GPTQ:** inverse Hessian H^{-1}; **AWQ:** protects 1% salient weights chosen by **activation magnitude**; **SmoothQuant:** migrates activation outliers to weights; **STE:** $\frac{\partial \text{round}(u)}{\partial u} = 1$.

Question ID	Core Concept Tested	The 10-Second Formula / Golden Rule
`LLM19`	Knowledge Distillation T^2 & Pruning	Multiply soft-target KL loss by T^2 to compensate for $1/T^2$ gradient shrinkage; **2:4 Sparsity** (2 zeros per 4 weights) gives $2\times$ Sparse Tensor Core speedup.
`LLM20`	Speculative Decoding & Two-Stage RAG	Speculative Decoding verifies γ draft tokens in 1 parallel Target forward pass (**100% lossless**); RAG uses **Bi-Encoder** (fast MIPS retrieval) $\rightarrow$ **Cross-Encoder** (joint attention reranking).